

路径规划

温程璐

单位：厦门大学

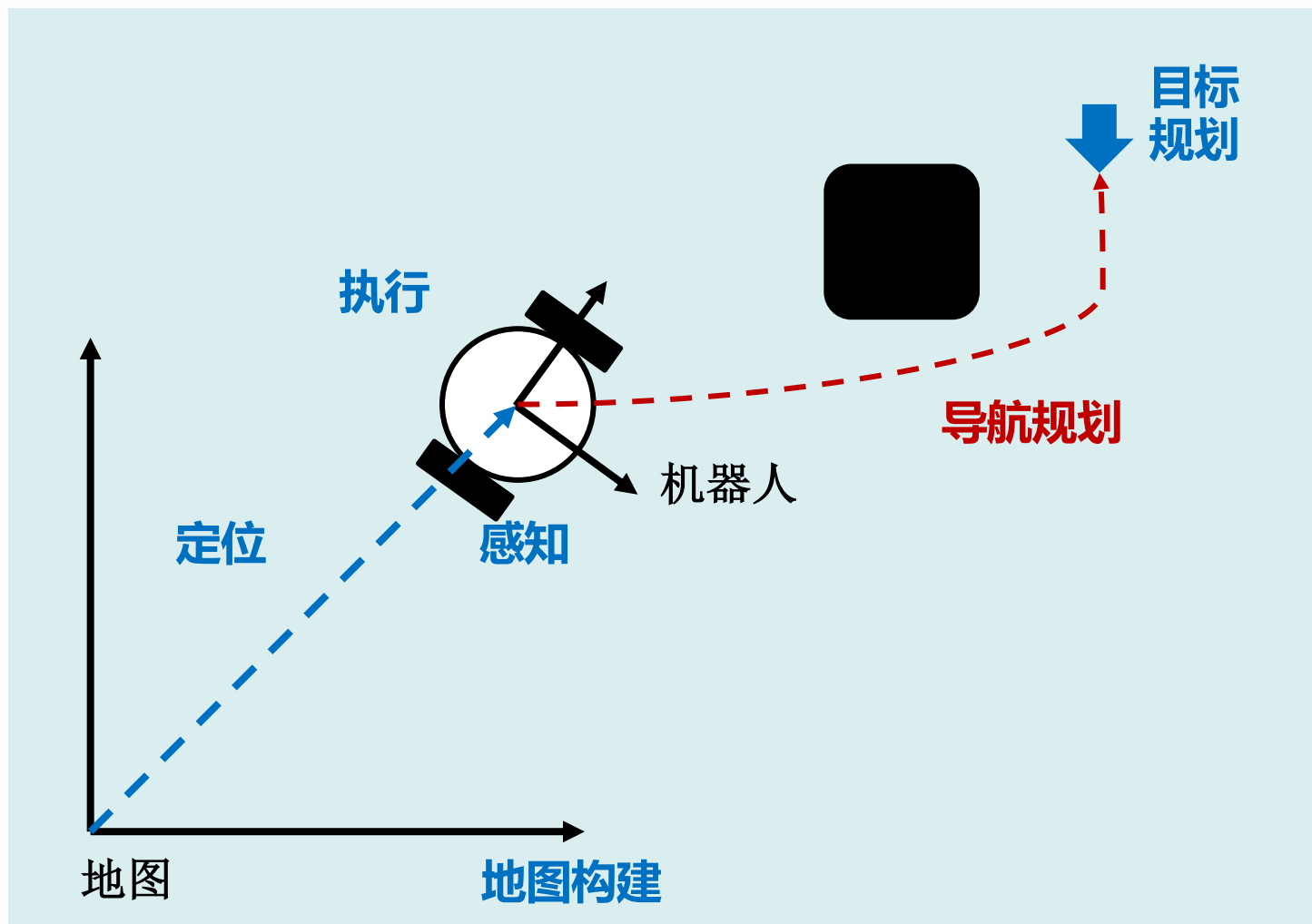
2023 年 9 月



部分课件内容参考自
浙江大学熊蓉老师

4.1 概述

导航规划

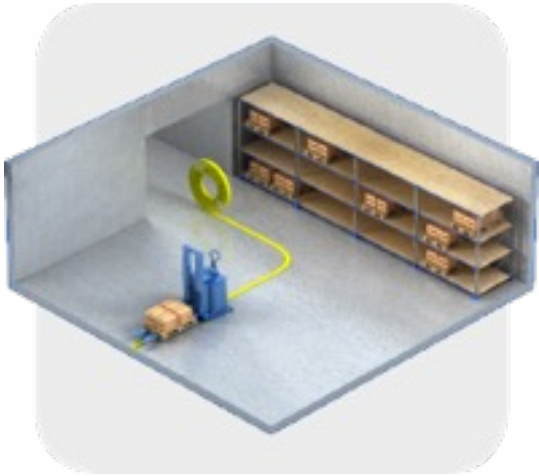


在给定环境的全局或局部知识以及一个或者一系列目标位置的前提下，使机器人能够根据知识和传感器感知信息高效可靠地到达目标位置

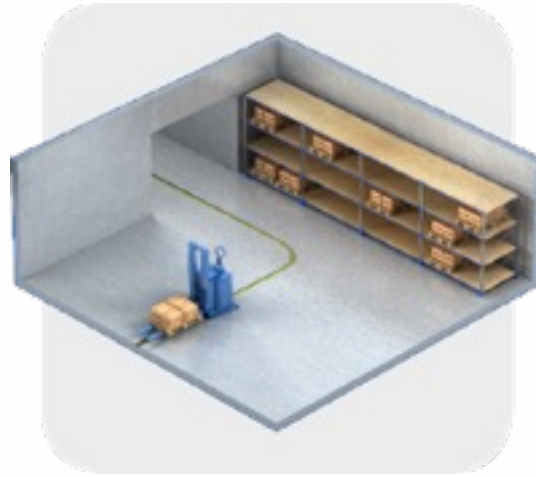
导航方式

- 固定路径导引：
 - 有人工标识导引
- 无轨导航：
 - 有人工标识导引的无固定路径（无轨）导航
 - 无标识导引的自然无轨导航

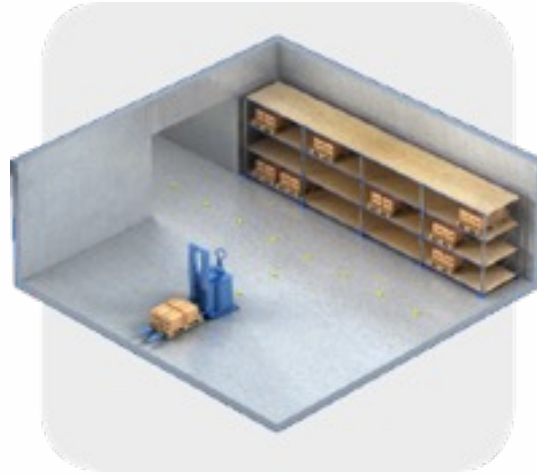
方式1：有人工标识导引的固定路径导航



磁条导航



磁感应线导航



磁钉导航



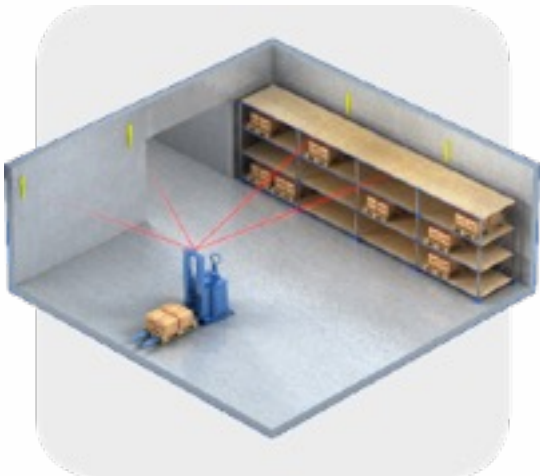
二维码导航



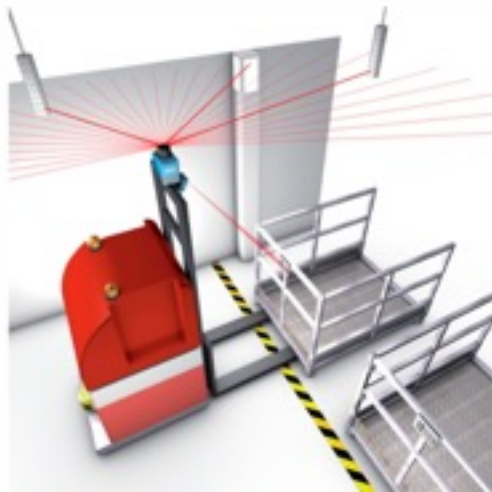
AGV: Automatic Guided Vehicle 自动导引车

- 优点：技术成熟、稳定可靠、价格优惠
- 缺点：需要施工和维护、路线无法调整、降低机器人自主适应能力

方式2：有人工标识导引的无轨导航



激光反射板导航



- 优点：技术成熟、路径可调
- 缺点：需要施工和维护、价格昂贵、被遮挡后效果下降

方式3：无人工标识导引的无轨导航



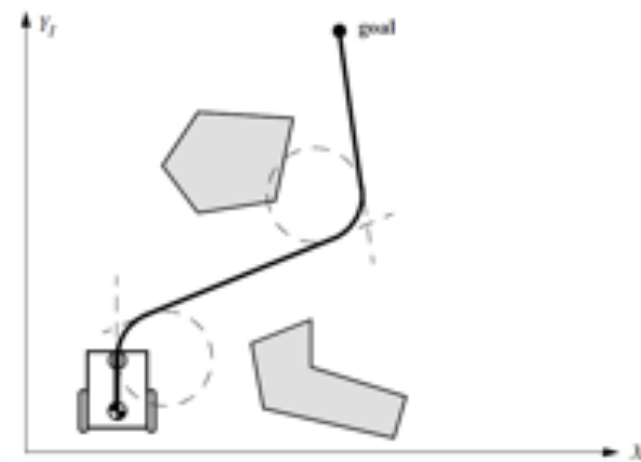
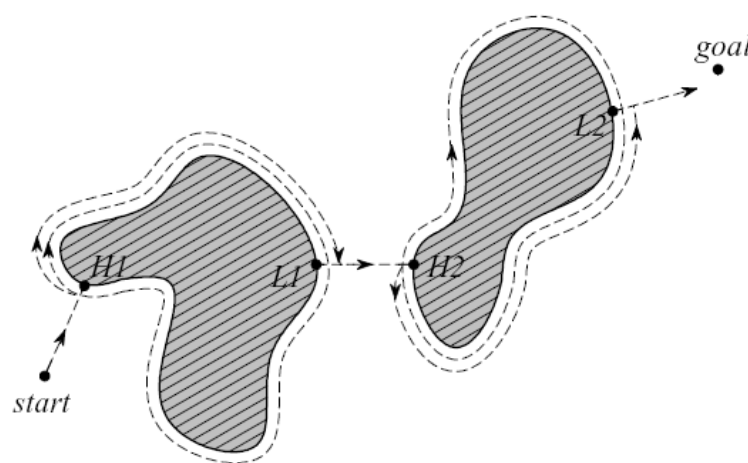
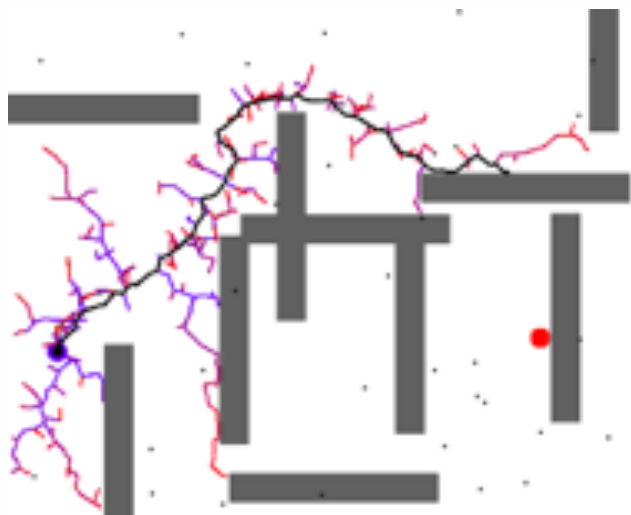
自然导航



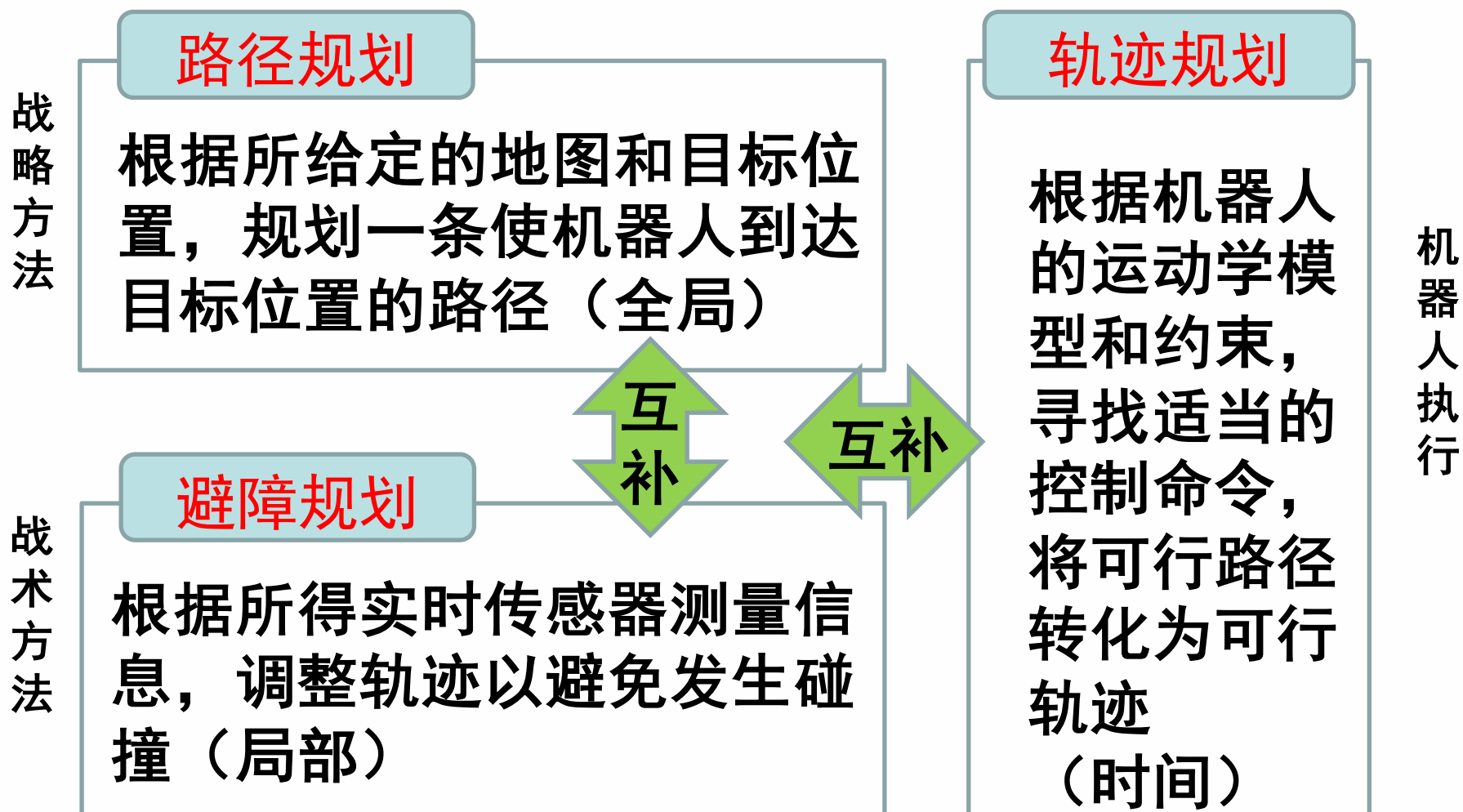
- 优点：无需施工、路径可调、精确定位、室内外通用
- 缺点：算法复杂，环境变化影响定位可靠性和稳定性

导航规划问题分解

- **路径规划**：根据所给定的地图和目标位置，规划一条使机器人到达目标位置的路径（只考虑工作空间的几何约束，不考虑机器人的运动学模型和约束）
- **避障规划**：根据所得到的实时传感器测量信息，调整路径/轨迹以避免发生碰撞
- **轨迹生成**：根据机器人的运动学模型和约束，寻找适当的控制命令，将可行路径转化为可行轨迹



路径规划、避障规划、轨迹规划三者关系



4.2 路径规划

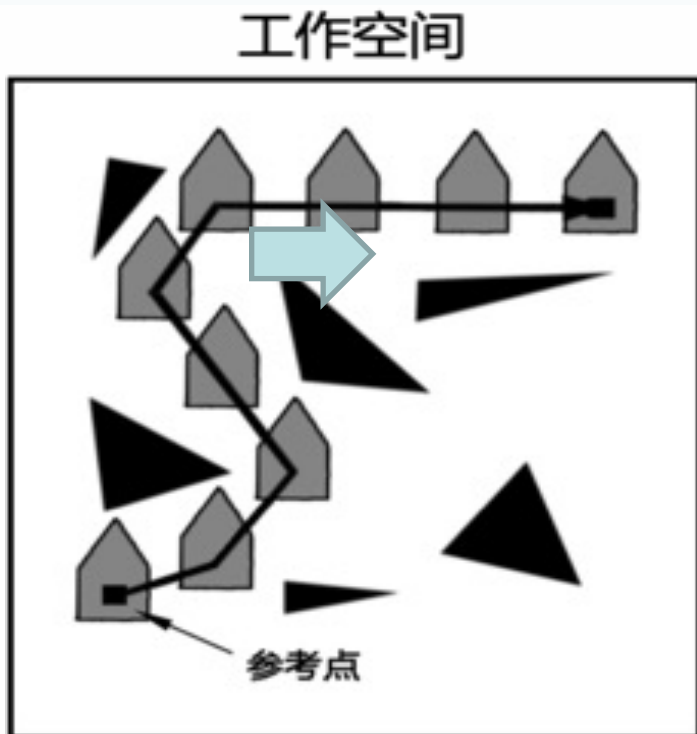
4.2.1 基本概念

路径规划

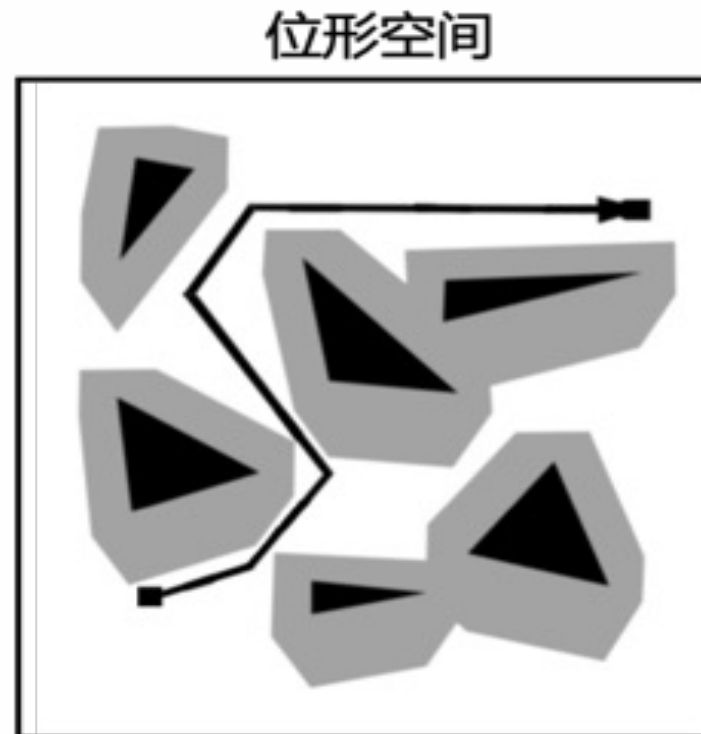
- 定义：根据所给定的地图和目标位置，规划一条使机器人到达目标位置的路径
- 通常简化为只考虑工作空间的几何约束
- 不考虑机器人的运动学模型和约束

工作空间与位形空间(C-Space)

工作空间：移动机器人上的参考点能达到的空间集合，机器人采用**位置**和**姿态描述**，并需考虑体积



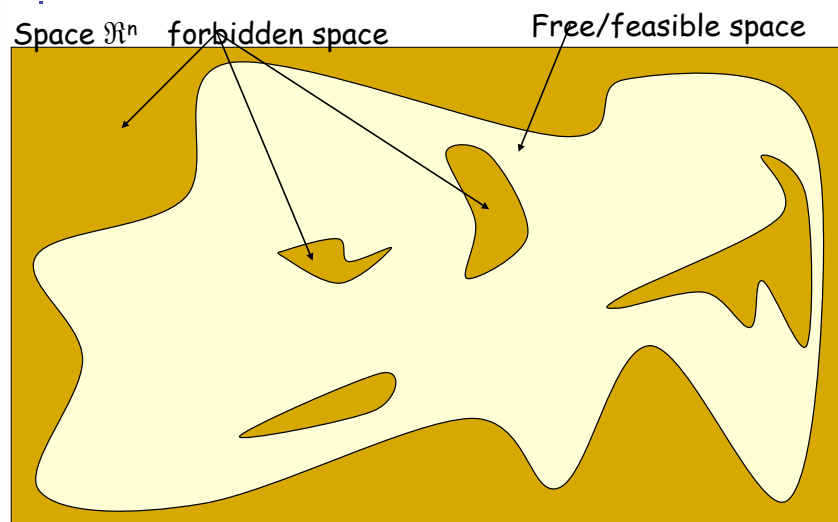
位形空间：机器人成为一个**可移动点**，不考虑姿态、体积和非完整运动学约束，仅做位置的描述和规划



忽略非完整约束对姿态的限制 障碍物按机器人半径进行膨胀

位形空间

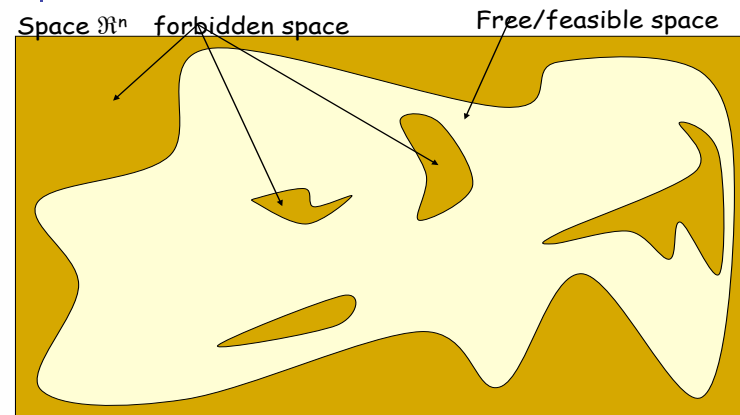
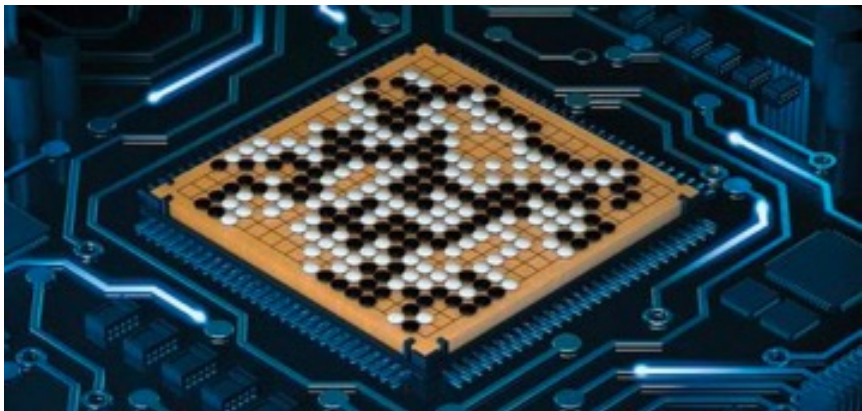
- 障碍物空间：
 - 不可行的位形集合
 - 在该空间中，机器人会与障碍物发生碰撞
- 自由空间：
 - 可行的位形集合
 - 在该空间中，机器人将无碰地安全移动



路径规划就是在自由位形空间中为机器人寻找一条路径，使其从初始位置运行到目标位置

路径规划方法需要具备完备性

- 完备性：对于问题的所有可能情况，当解存在时，能够在有限的时间内找到解；解不存在的时候，能够有限时间返回失败
- 路径规划算法挑战：
 - 在连续空间内搜索，难以保证实时性要求，难以达到完备性
- 保证完备性解决方案：
 - 对连续位形空间做离散化，达到近似完备



路径规划方法需要具备完备性

确保完备性的方法

- 基本思路：对空间作离散化
 - (1) 分辨率完备的路径规划：通过解析性离散化，确保获得可行解
 - 行车图法：基于障碍物几何形状分解姿态空间
 - 单元分解法：区分空闲单元和被占单元
 - 势场法：根据障碍物和目标对空间各点施加虚拟力
 - (2) 概率完备的路径规划：基于概率进行随机采样离散化，使获得解的概率趋近于1
 - PRM (Probabilistic RoadMaps)
 - RRT (Rapid-Exploring Random Trees)

路径规划方法需要具备完备性

以上方法都用于构建连通图，下一步是从图中搜索最优路径

(1) 精确最优搜索法：深度优先法、宽度优先法

(2) 近似最优搜索法

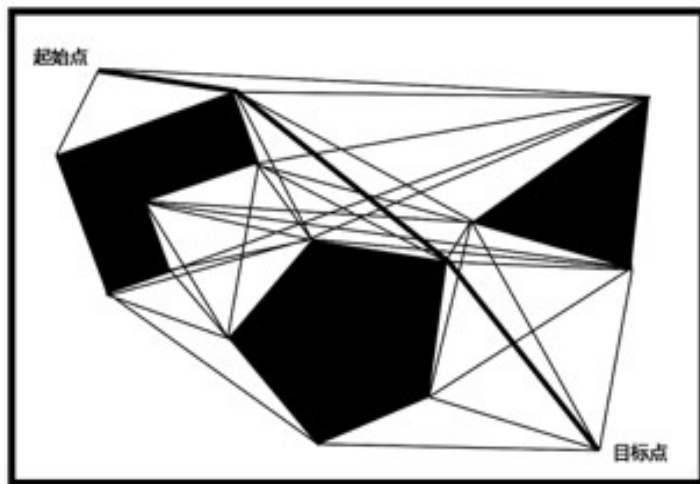
- 启发式搜索法： A^* ， D^*
- 准启发式搜索算法：退火、进化和蚁群优化等

4. 2. 2 分辨率完备的路径规划方法

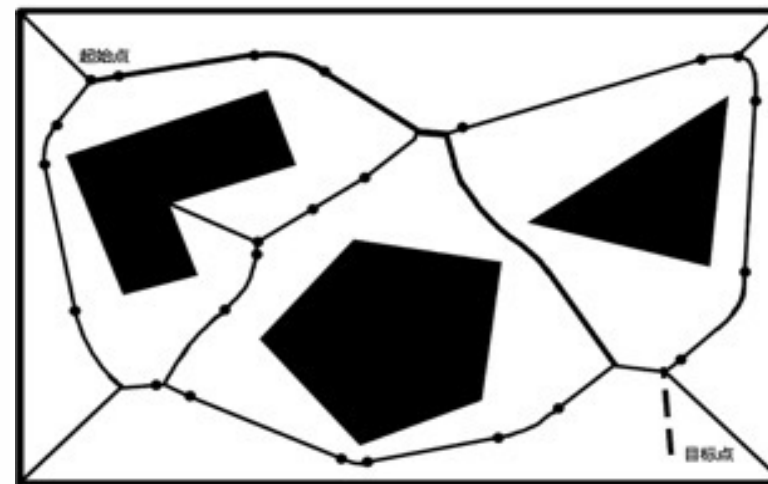
- 行车图法： 基于障碍物几何形状分解姿态空间
- 单元分解法： 区分空闲单元和被占单元
- 势场法： 根据障碍物和目标对空间各点施加虚拟力

1. 行车图法 (Road map)

- 基本思想：基于障碍物几何形状分解位形空间，将自由空间的连通性用一维曲线的网格表示，在加入起始点和目标点后，在该一维无向连通图中寻找一条无碰路径
- 无向图 $R=(N,E)$ ， N 为节点（路径点）集合， E 为两节点存在一条可行路径。
- 难点：所构建的道路确保机器人在自由空间中，又要最小化整个道路数量
- 两种构建行车图的典型方法：



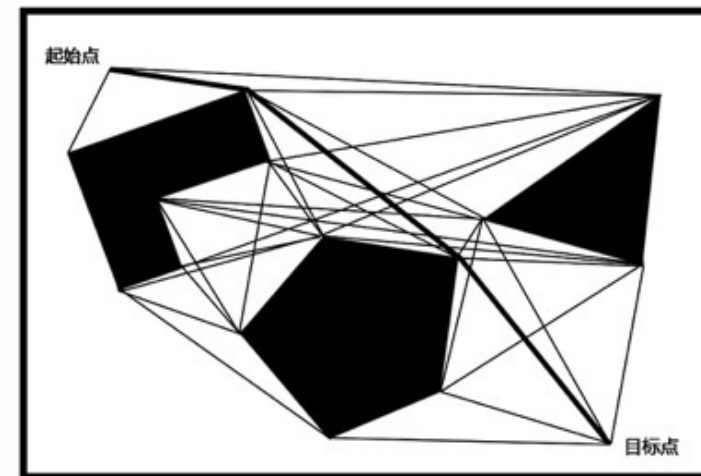
可视图 (Visibility graph)



Voronoi diagram

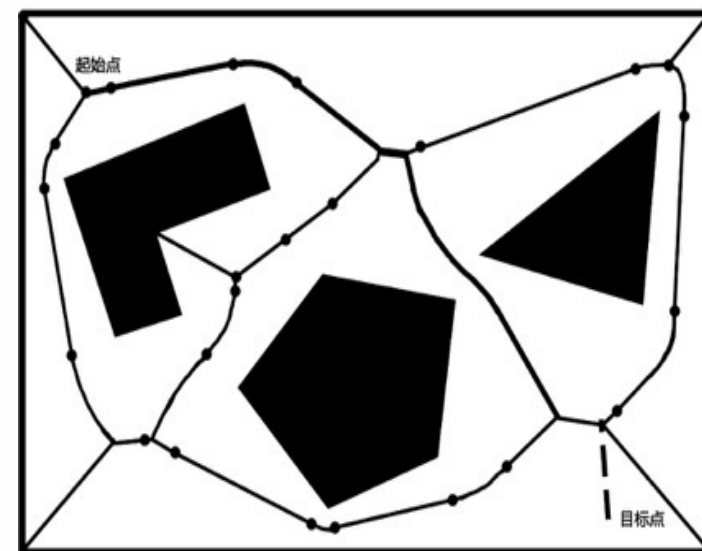
1.1 可视图法

- 可视图由所有连接可见顶点对的边组成
 - 可见：顶点之间无障碍物
 - 初始位置和目标位置也作为顶点
- 优点：
 - 简单，特别是当环境地图用多边形描述物体时
 - 可得到在路径长度上最优的解
- 缺点：
 - 所得路径过于靠近障碍物，不够安全
- 常用的解决方法：
 - 以远大于机器人半径的尺寸膨胀障碍物，但容易造成可行路径的消失
 - 在路径规划后修改所得路径，使其与障碍物保持一定的距离



1.2 Voronoi diagram法

- 基本思想：取障碍物之间的中间点，以最大化机器人和障碍物之间的距离
- 构建方法：
 - 对于自由空间中每一点，计算它到最近障碍物的距离
 - 在垂直于二维空间平面的轴上用高度表示该点到障碍物的距离，类似于画直方图
 - 当某个点到两个或多个障碍物距离相等时，其距离点处出现尖峰，Voronoi图由连接这些尖峰点的边组成
- 优点：安全性高
- 缺点：计算复杂、路径长度较可视图法长、不适用于短距离定位传感器



2. 单元分解法

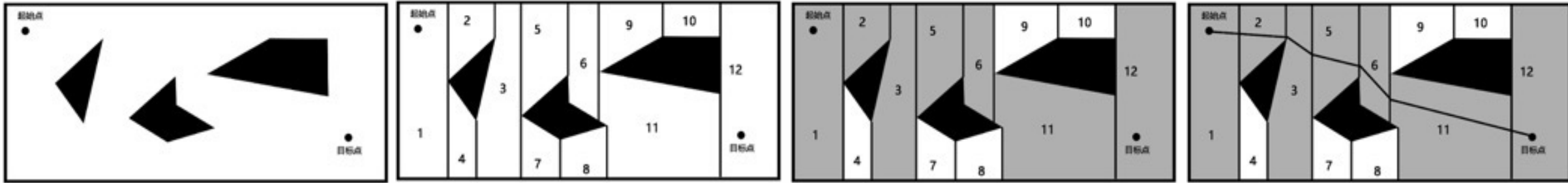
- 基本思想

- 首先，将位形空间中的自由空间分为若干的小区域，每一个区域作为一个单元，以单元为顶点、以单元之间的相邻关系为边构成一张连通图；
- 其次，在连通图中寻找包含初始姿态和目标姿态的单元，搜索连接初始单元和目标单元的路径；
- 最后，根据所得路径的单元序列生成单元内部的路径

- 主要方法

- 精确单元分解
- 近似单元分解

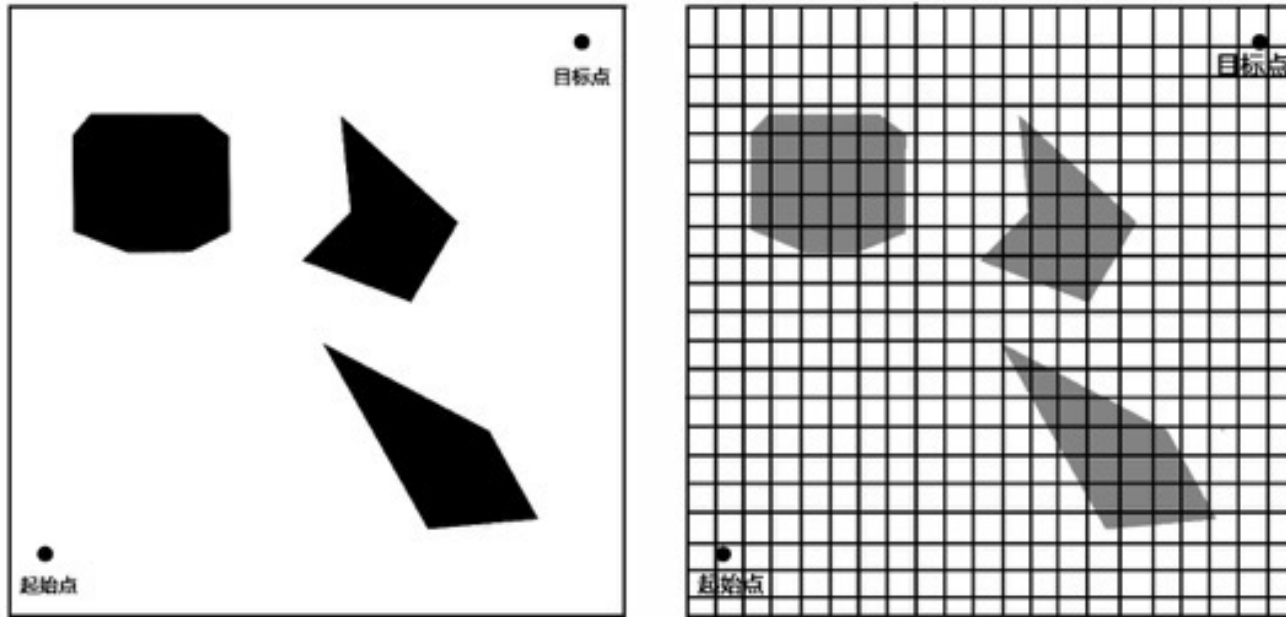
2.1 精确单元分解



- 单元边界严格基于环境几何形状分解，所得单元空闲或者被占用
- 优点：
 - 机器人不需要考虑它在每个空闲单元中的具体位置，只需要考虑如何从一个单元移动到相邻的空闲单元
 - 单元数与环境大小无关
- 缺点：计算效率极大地依赖于环境中物体的复杂度
物体形状对分解提出很大挑战，很少使用

2.2 近似单元分解

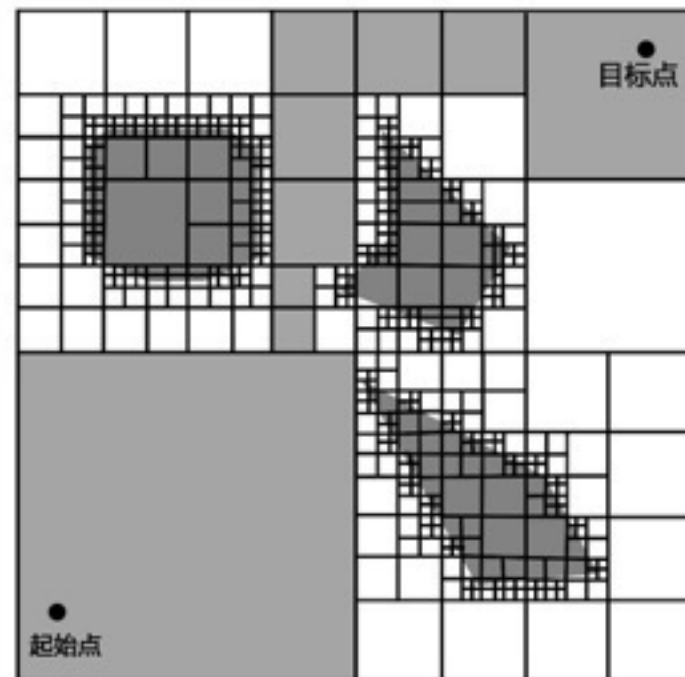
- 栅格表示法，将环境分解成若干个大小相同的栅格



并不是每个单元都是完全被占或者完全空闲的，因此分解后的单元集合是对实际地图的一种近似

- 优点：非常简单，与环境的疏密和物体形状的复杂度无关
- 缺点：对存储空间有要求，地图越大，单元越多

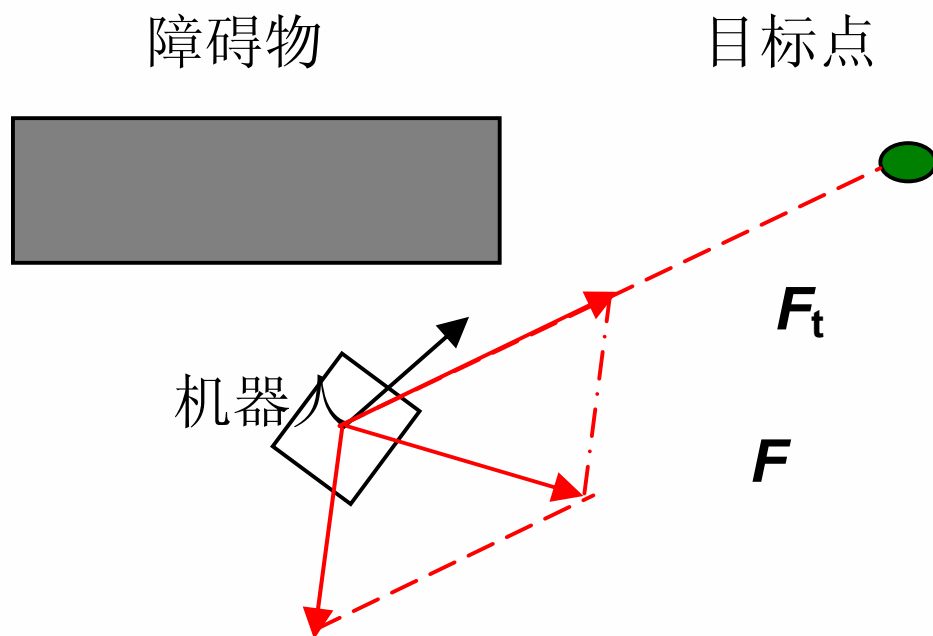
可变大小的近似单元分解



四叉树表示法：递归地把环境分为4个大小相等的子区域。
直到每个区域中所包含的基本元素全为0或全为1

3. 人工势场法

- 基本思想：
 - 目标点对机器人产生吸引力，障碍物对机器人产生排斥力
 - 所有力的合成构成机器人的控制律

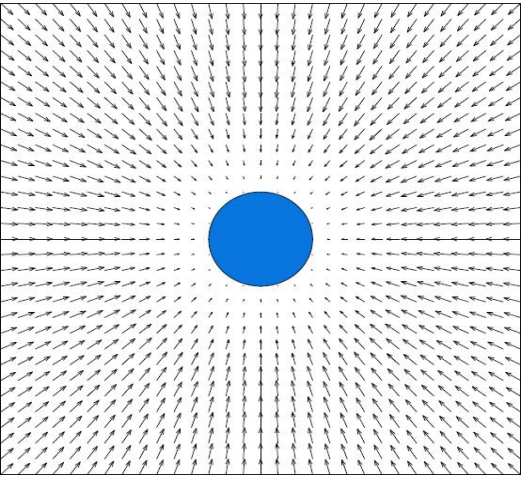


3. 人工势场法

和目标距离大时，
用平方函数会吸力
过大，大于排斥力
，碰撞障碍物

- 步骤1：构建人工势场(Artificial Potential Field)

- 目标点：吸引势场



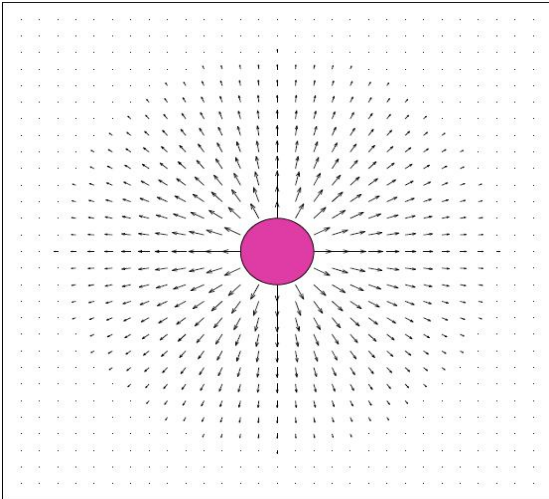
$$U_{att}(\mathbf{x}) = \begin{cases} K_a |\mathbf{x} - \mathbf{x}_d|^2 & |\mathbf{x} - \mathbf{x}_d| \leq d_a \\ K_a (2d_a |\mathbf{x} - \mathbf{x}_d| - d_a^2) & |\mathbf{x} - \mathbf{x}_d| > d_a \end{cases}$$

平方函数

分段函数

线性函数

- 障碍物：排斥势场



分段函数

$$U_{rep}(\mathbf{x}) = \begin{cases} \frac{1}{2} K_r \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right)^2 & \rho \leq \rho_0 \\ 0 & \rho > \rho_0 \end{cases}$$

越接近障碍物，势
场由0变为正值，
并不断增大

ρ 被评估点和障碍物点之间的距离

ρ_0 预定义距离阈值

K_a 玻□亥
 $\mathbf{x}$ 玻□□摧□
 $\mathbf{x}_d$ 玻□□□
 d_a 玻□□□度

3. 人工势场法

- 步骤2：根据人工势场计算力
 - 对势场求偏导数

$$\text{吸引力 } F_{att}(\mathbf{x}) = -\nabla U_{att}(\mathbf{x}) = \begin{cases} -2K_a(\mathbf{x} - \mathbf{x}_d) & |\mathbf{x} - \mathbf{x}_d| \leq d_a \\ -2K_a d_a \frac{\mathbf{x} - \mathbf{x}_d}{|\mathbf{x} - \mathbf{x}_d|} & |\mathbf{x} - \mathbf{x}_d| > d_a \end{cases}$$

$$\text{排斥力 } F_{rep}(\mathbf{x}) = -\nabla U_{rep}(\mathbf{x}) = \begin{cases} K_r \left(\frac{1}{\rho} - \frac{1}{\rho_0} \right) \frac{1}{\rho^2} \frac{\partial \rho}{\partial \mathbf{x}} & \rho \leq \rho_0 \\ 0 & \rho > \rho_0 \end{cases}$$

$$\frac{\partial \rho}{\partial \mathbf{x}} = \left(\frac{\partial \rho}{\partial x} \quad \frac{\partial \rho}{\partial y} \right)^T = \frac{\mathbf{x} - \mathbf{x}_0}{\rho}$$

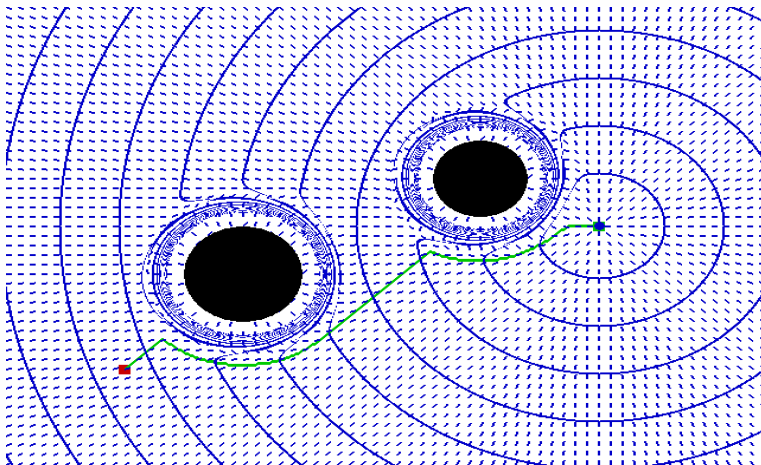
$\mathbf{x}_0$ 玻璃 ☐ ☐ ☐ ☐ ☐ 书 ☐ 柳 ☐

3. 人工势场法

- 步骤3：计算合力，并进而由力计算得到控制律

$$\begin{aligned} F(\mathbf{x}) &= -\nabla U(\mathbf{x}) \\ &= -\nabla U_{att}(\mathbf{x}) - \nabla U_{rep}(\mathbf{x}) \\ &= F_{att}(\mathbf{x}) + F_{rep}(\mathbf{x}) \end{aligned}$$

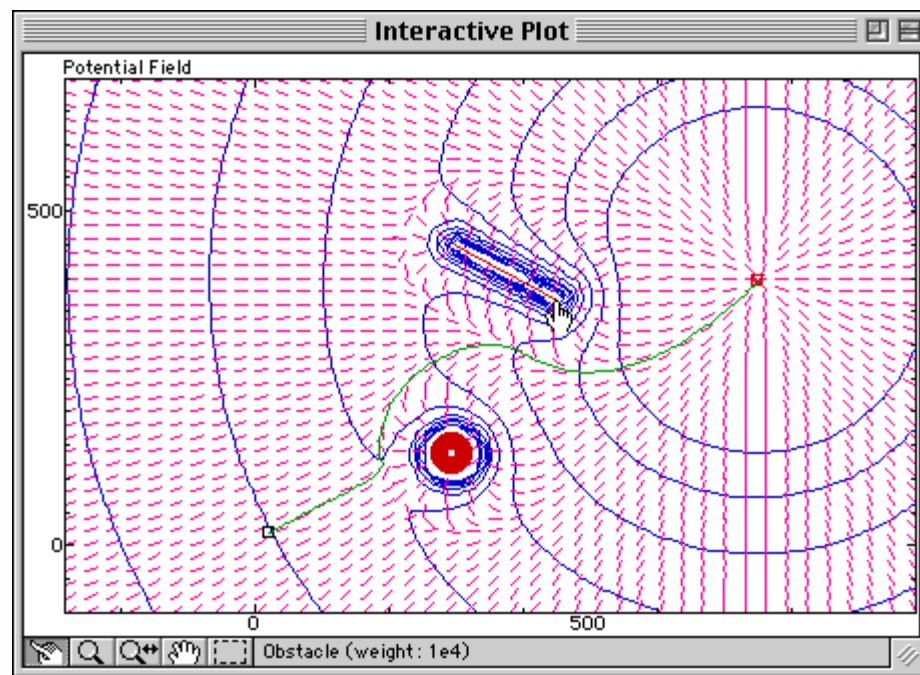
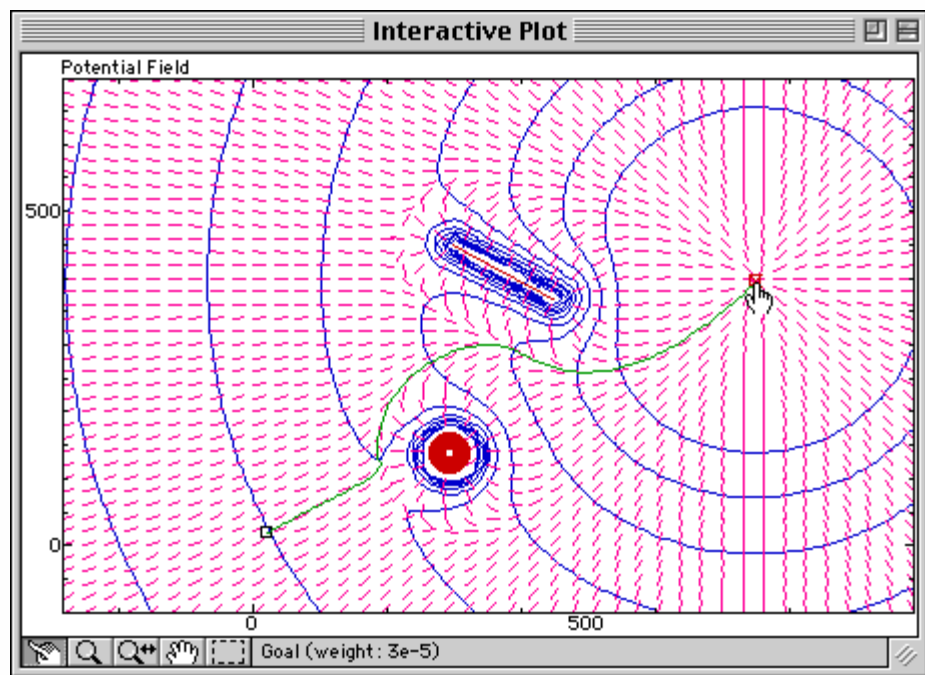
力的方向就是机器人运动方向，大小可以对应加速度控制，使得机器人避开障碍物，移动向目标



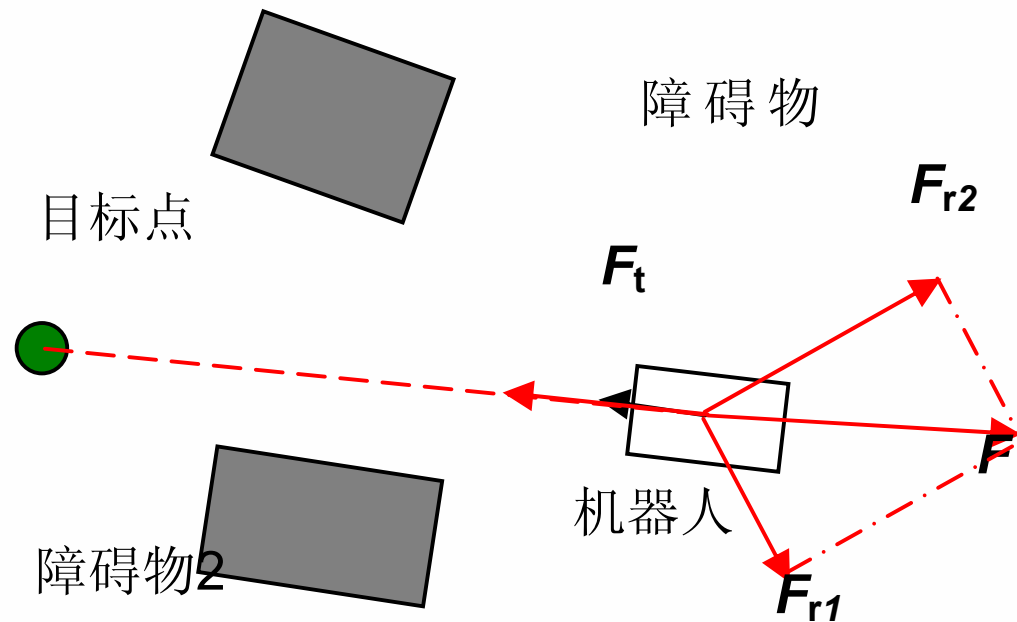
- 小蓝点为目标点，蓝色椭圆为障碍物
- 每个点的线段表示该点力的方向（势场变化方向）
- 机器人是受人工势场影响的一个点，沿着势场方向就可以避开障碍物达到目标点

3. 人工势场法

- 不仅是一种路径规划方法，所构建的势场也构成了机器人的控制律
- 目标或障碍物运动时，势场变化
- 能够较好地适应目标变化和环境中的动态障碍物，可以作为实时避障算法



3. 人工势场法



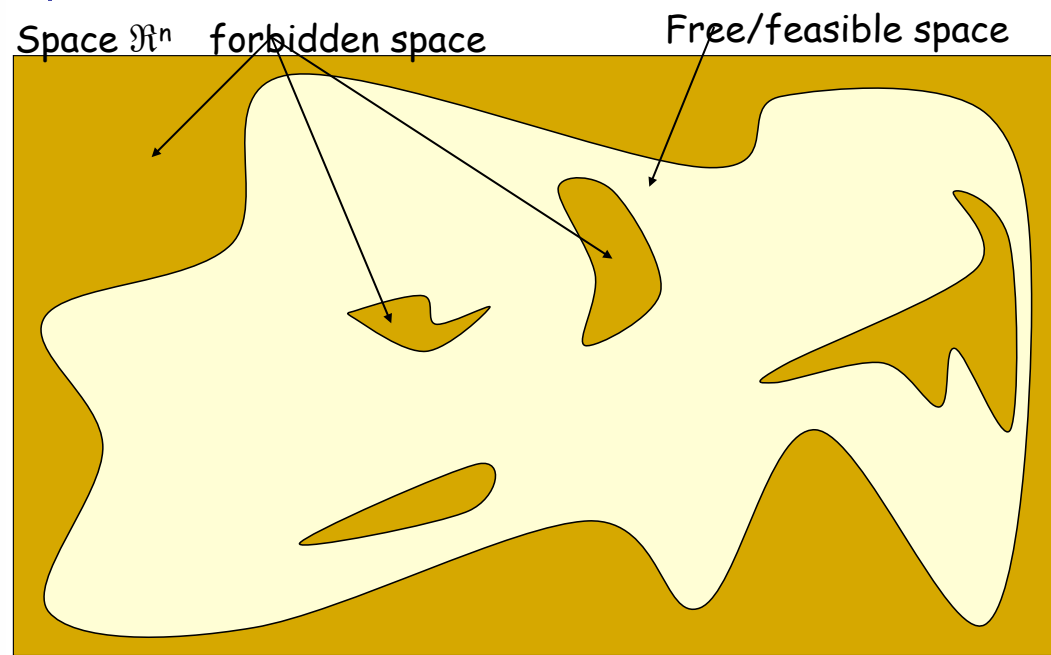
- 缺点：存在局部最小，容易产生振荡和死锁

4.2. 概率完备的连通图构建

- PRM (Probabilistic RoadMaps) 基于随机采样构建行车图的概率行车图法
- RRT (Rapid-Exploring Random Trees) 基于随机采样构建搜索树的快速扩展随机树法

1. PRM(Probabilistic Roadmap)

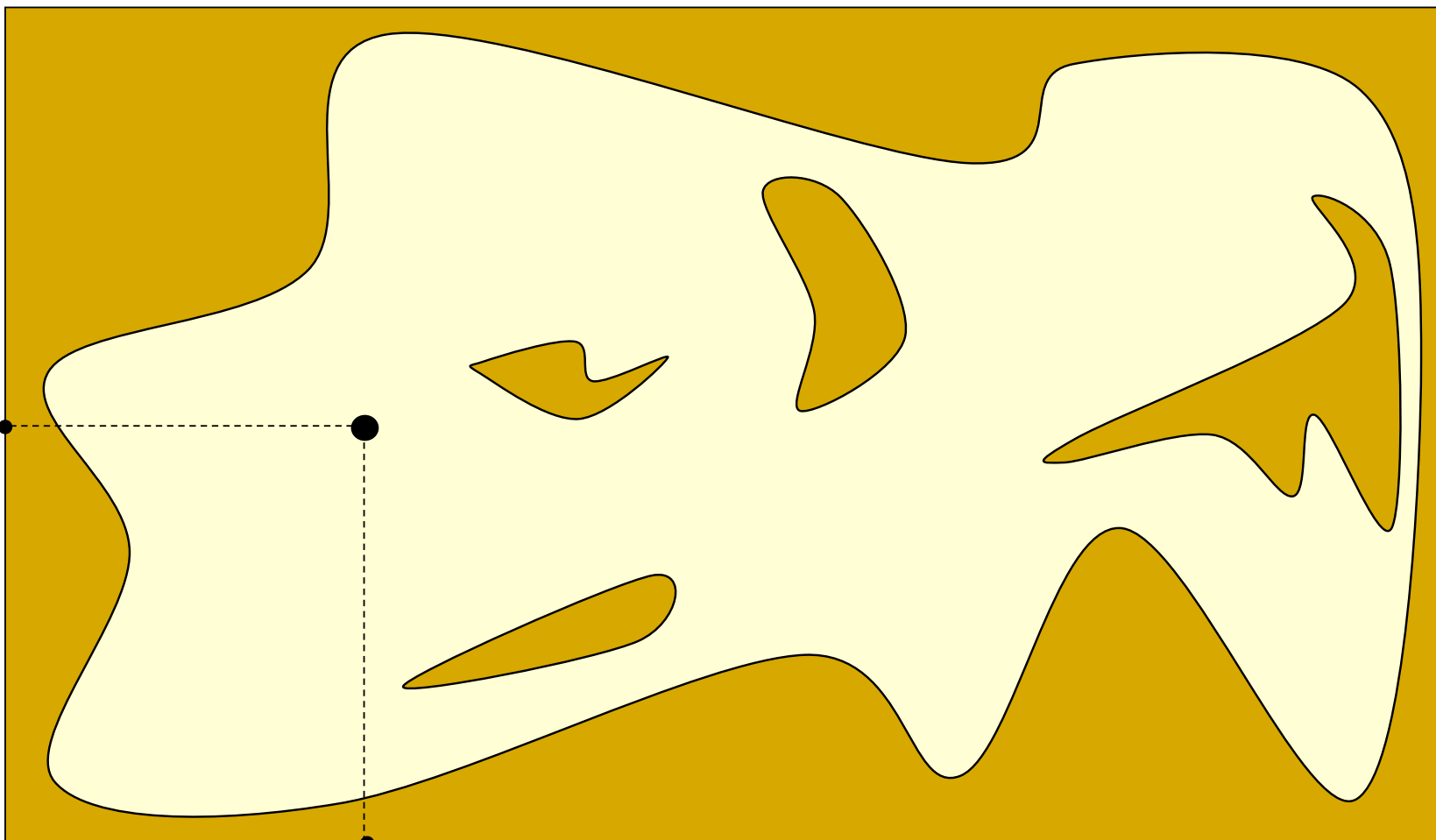
- 分辨率完备的路径规划方法适合于低维情况和简单障碍物
- 1996年斯坦福大学机器人实验室提出PRM方法
- 通过随机采样和碰撞检测找到自由位形空间中的路径点和无碰路径，构建连通图
- (1) 构建步骤：获得合理的连通图，均匀覆盖整个空间，确保最难区域有节点
- (2) 扩张步骤：根据启发式评估准则，在连通图寻找位形空间困难区域的节点，并扩张



1. PRM(Probabilistic Roadmap)

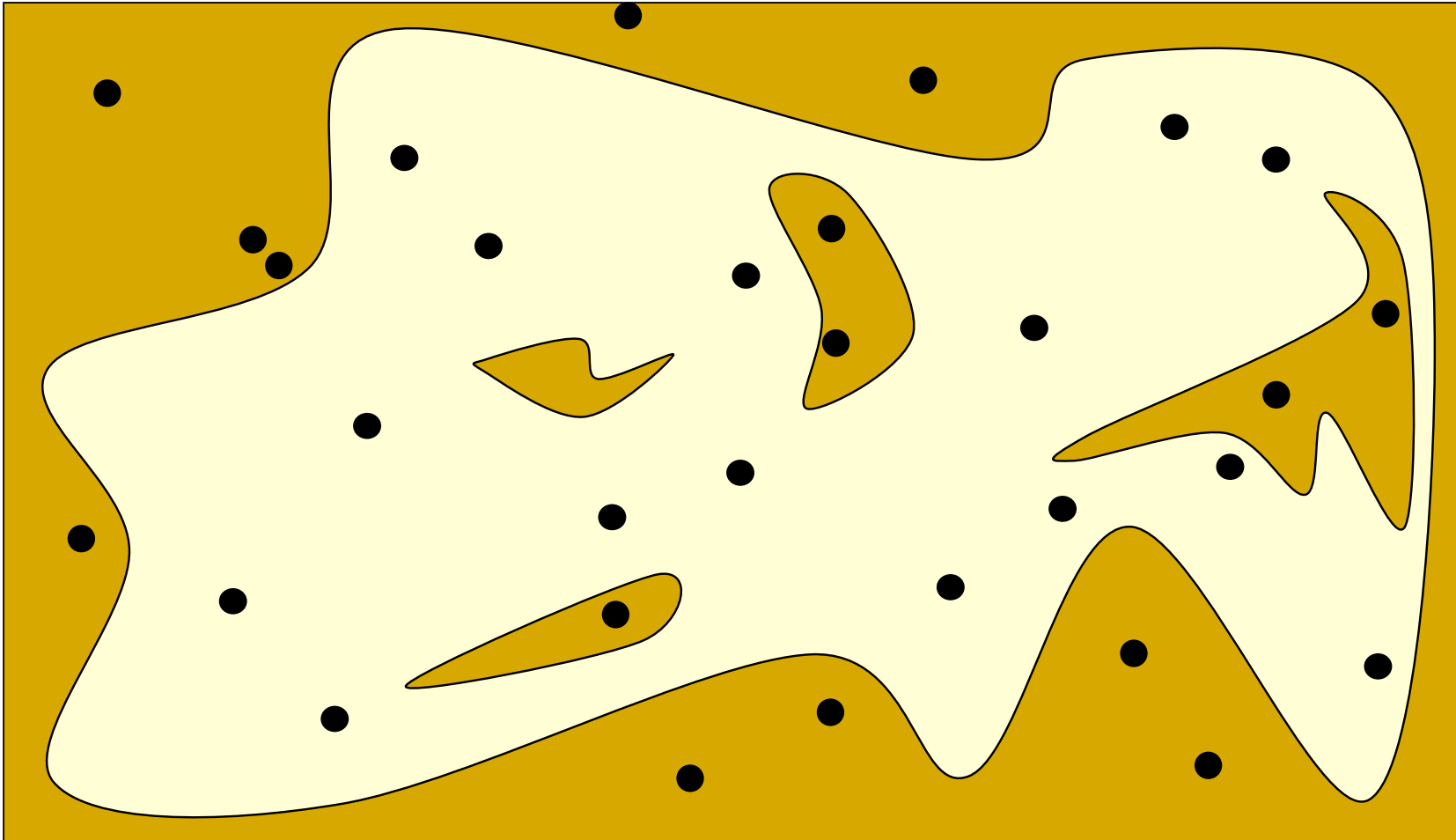
基于随机采样构建行车图的概率行车图法

在位形空间坐标系中随机取点



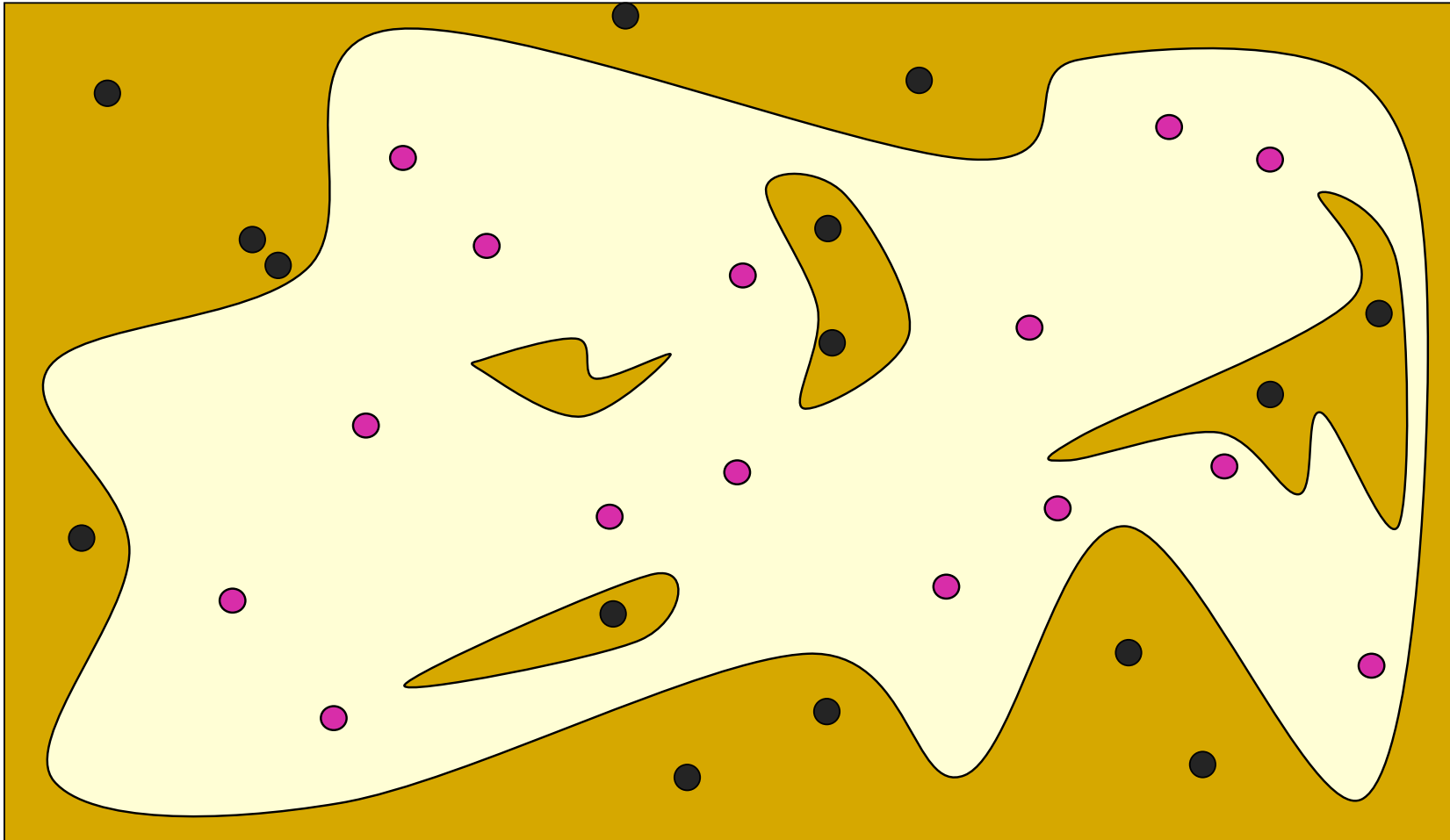
1. PRM(Probabilistic Roadmap)

在位形空间坐标系中随机取点



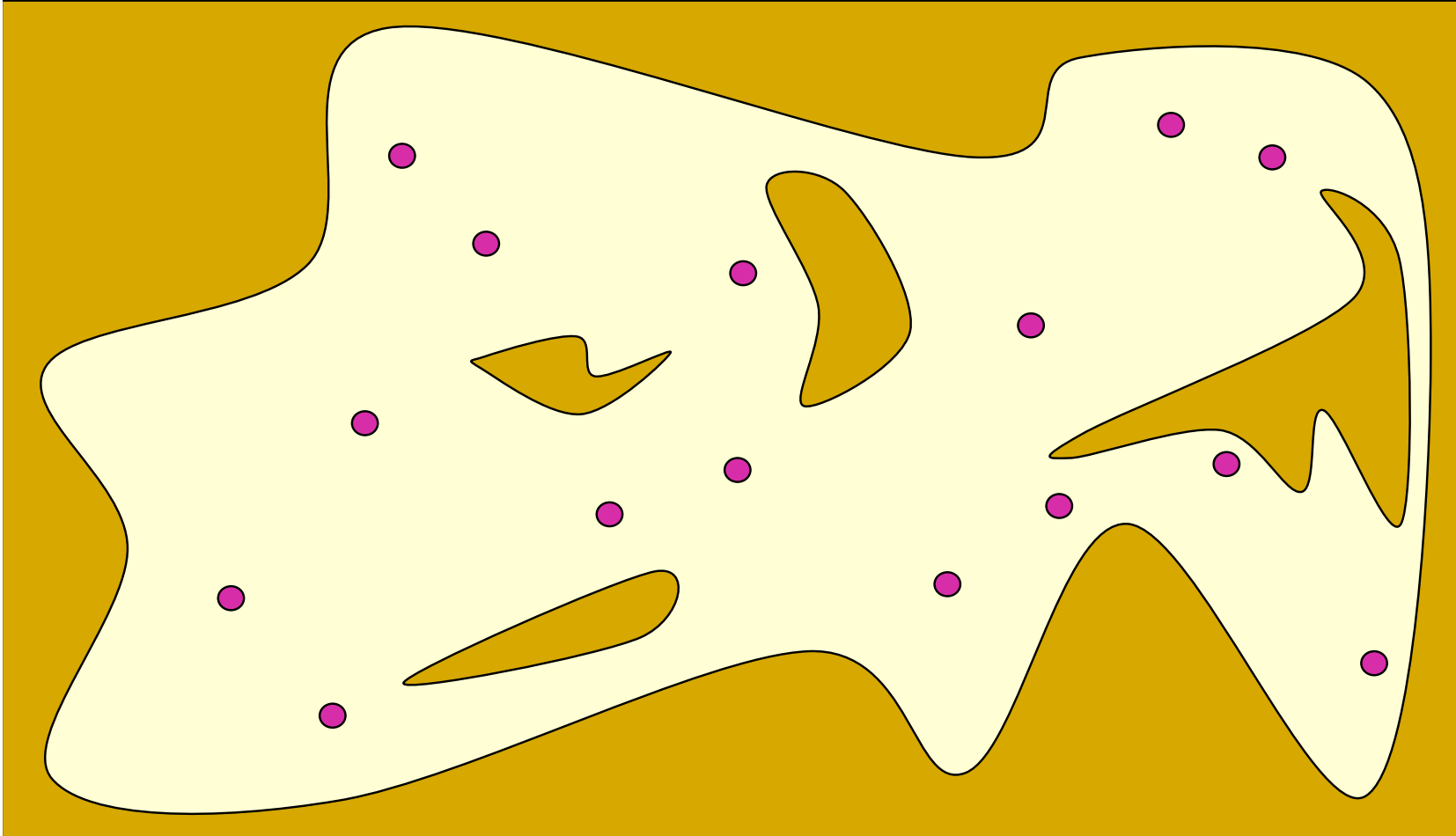
1. PRM(Probabilistic Roadmap)

对采样的姿态进行碰撞检测



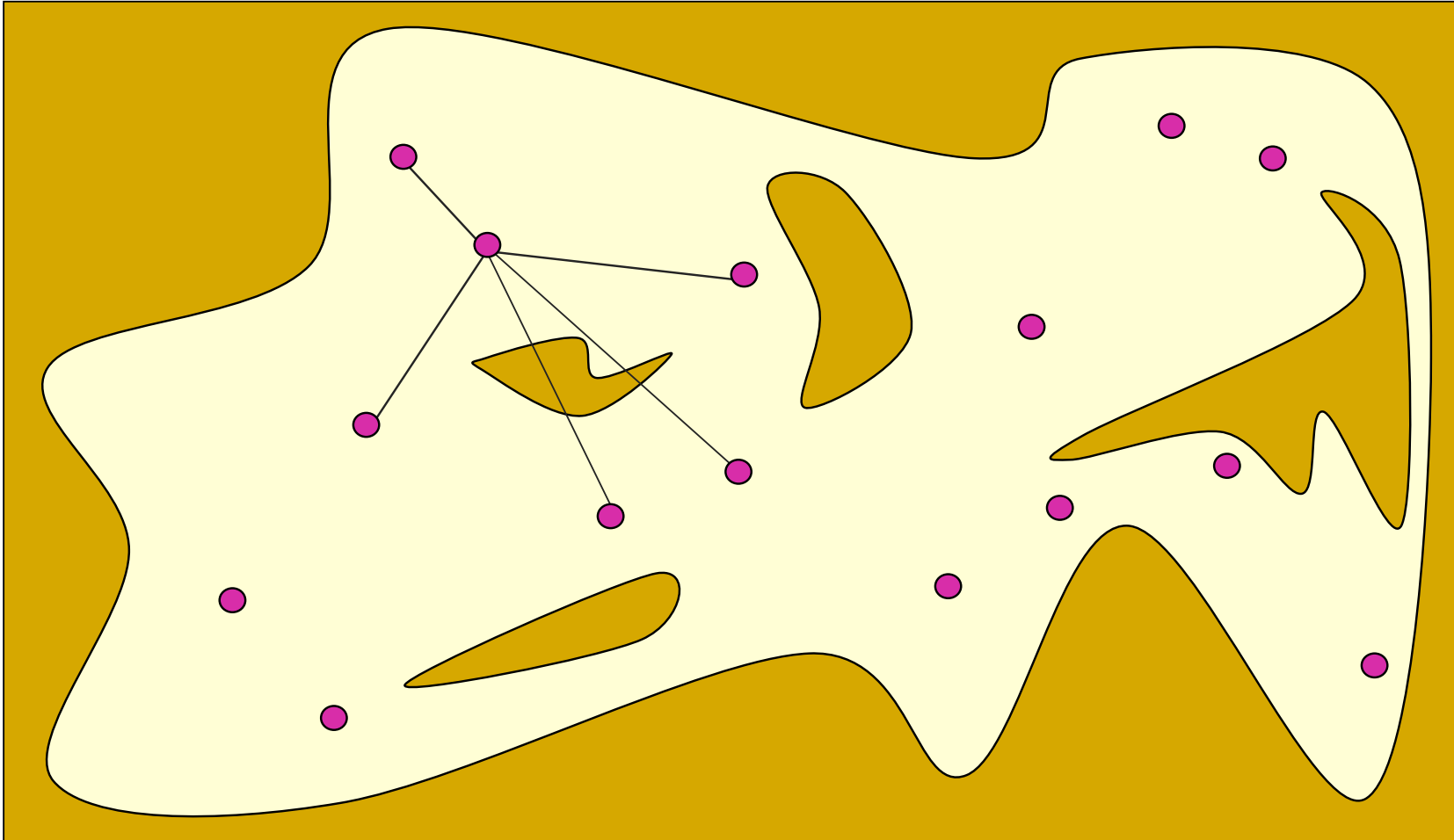
1. PRM(Probabilistic Roadmap)

无碰撞姿态成为图节点



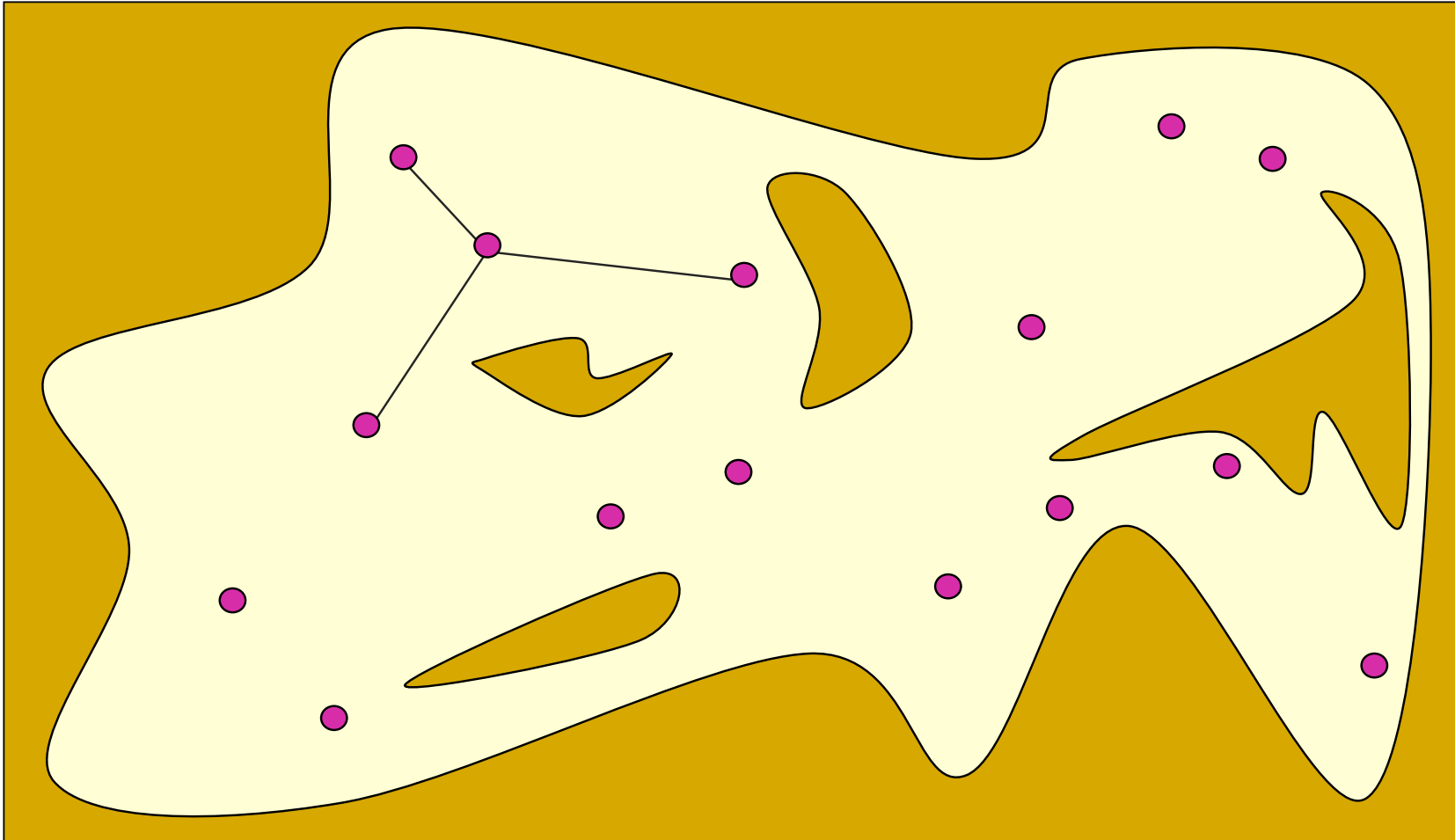
1. PRM(Probabilistic Roadmap)

每个图节点和其最近相邻的k个节点直线连接



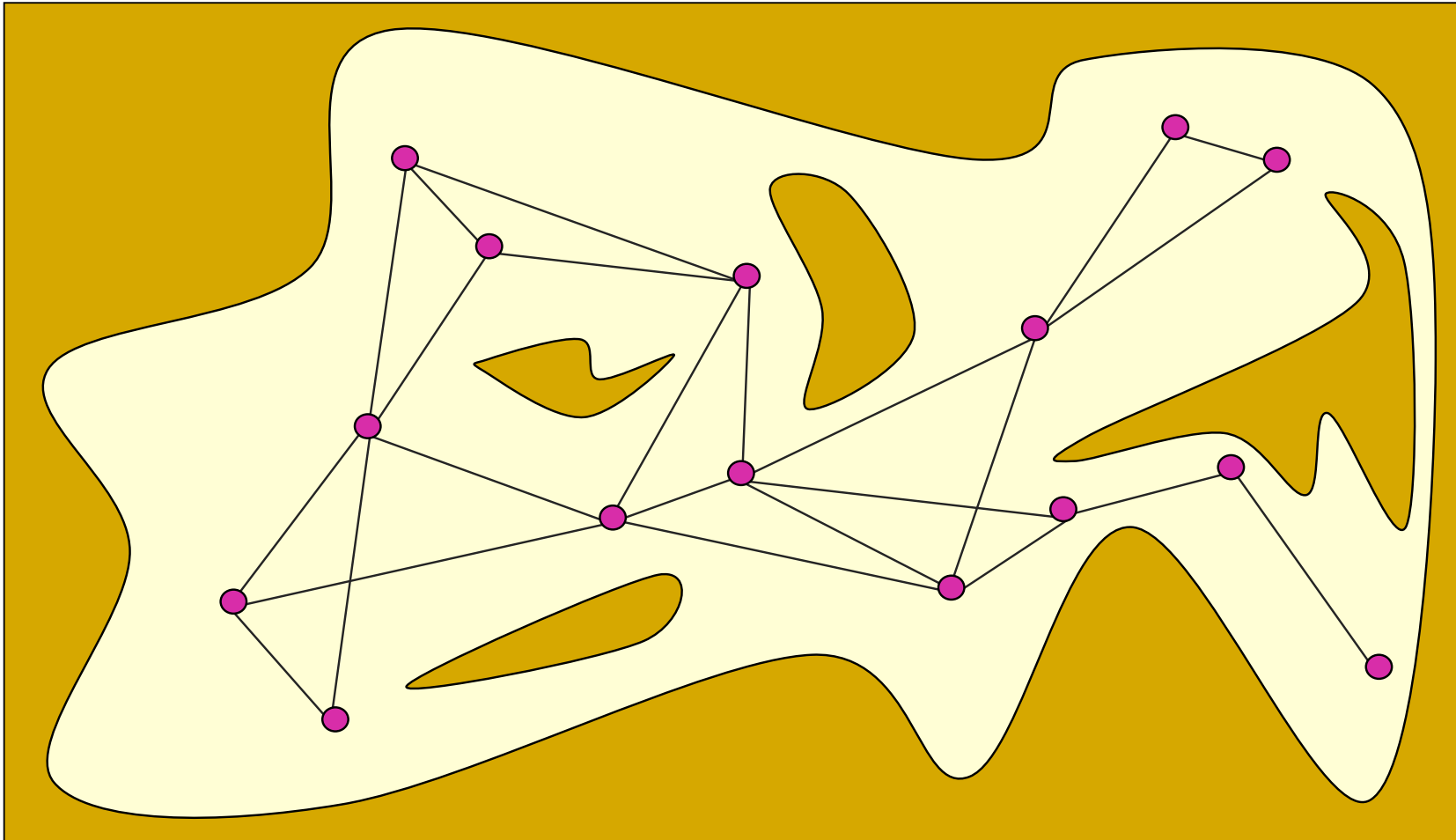
1. PRM(Probabilistic Roadmap)

保留无碰路径为图的边



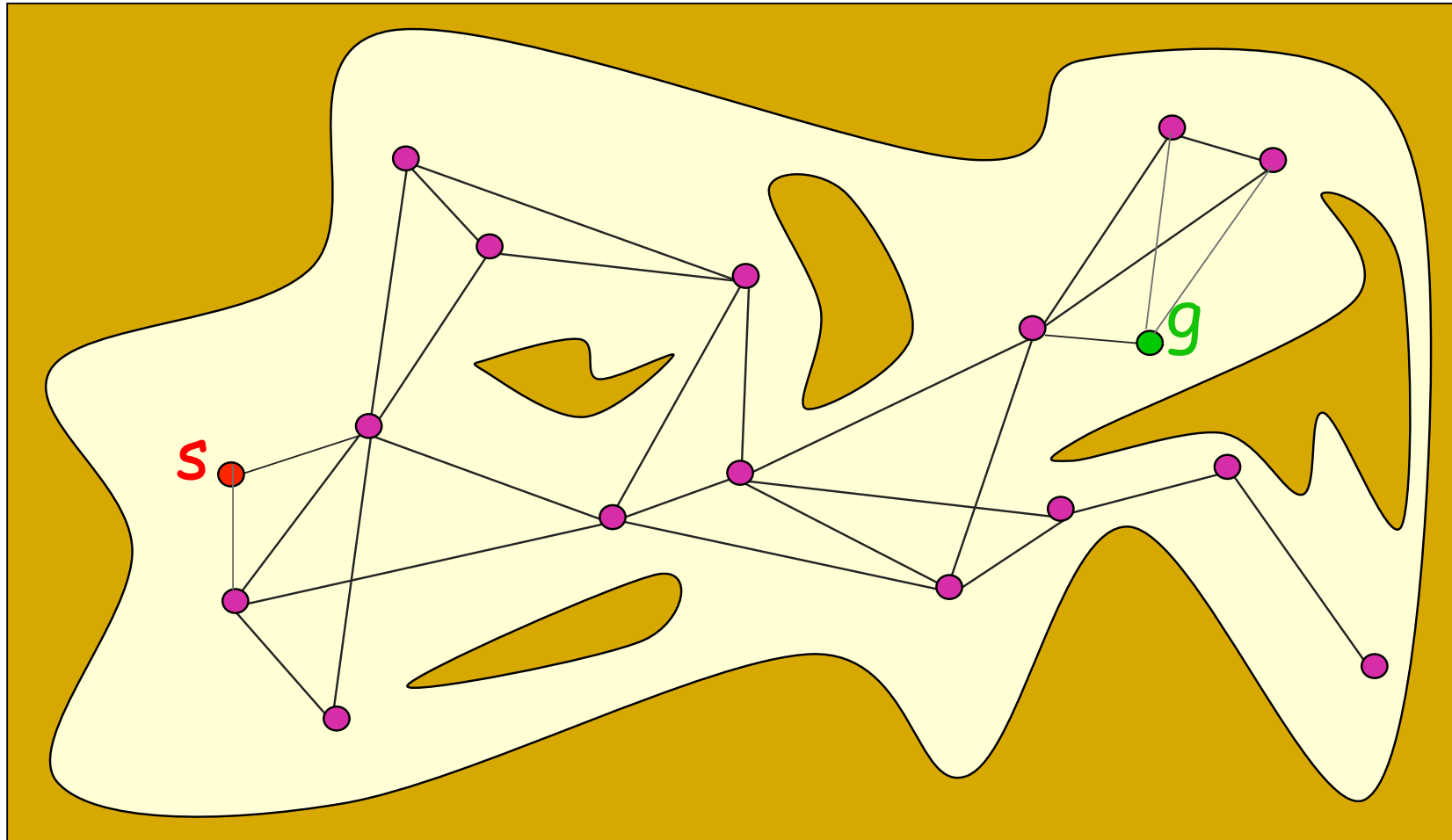
1. PRM(Probabilistic Roadmap)

构成自由位形空间中的Roadmap



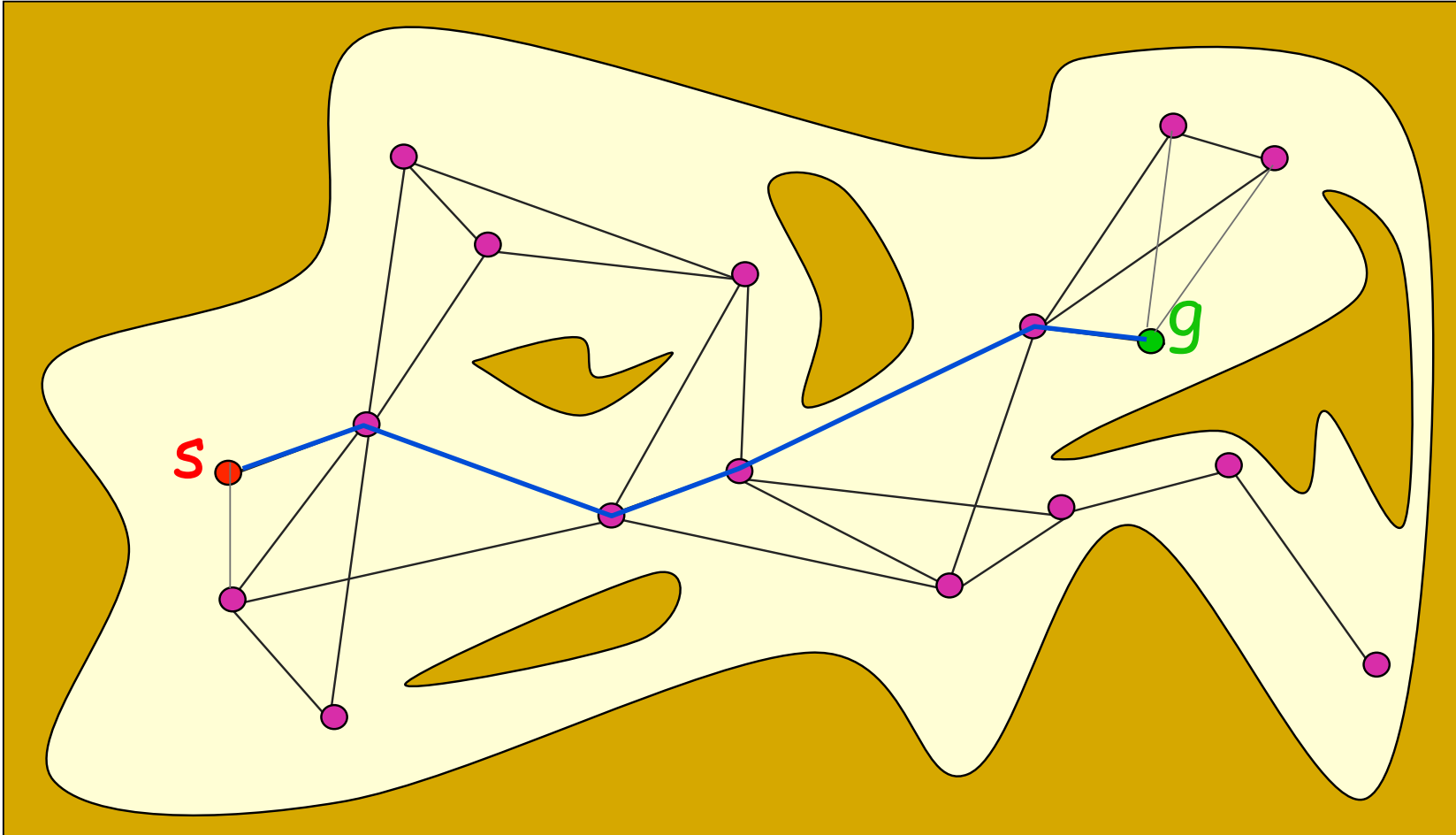
1. PRM(Probabilistic Roadmap)

加入起始点和终止点



1. PRM(Probabilistic Roadmap)

在PRM中搜索一条从起始点到终止点的路径

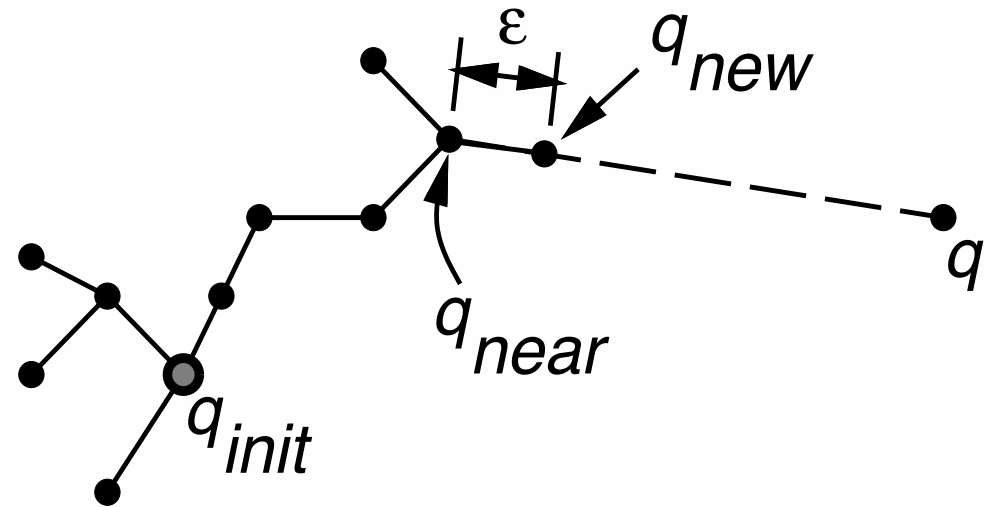


PRM

- 优点：
 - 简化了对环境的解析计算，可以快速构建得到行车图
 - 适用于高维度自由位形空间中的规划
 - 是一个近似完备的路径规划方法
- 缺点：
 - 对自由空间连通性表达的完整性依赖于采样次数
 - 从算法通用性上来讲难以评估需要多少时间做充分采样
 - 不考虑机器人执行的可行性

2. RRT(Rapid-Exploring Random Tree)

- 1998年由美国爱荷华州立大学Steven M.Lavalle教授提出
- 基本思想：
 - 连通图采用树的形式，以起始点作为树的根节点
 - 采用在空间中随机采样、连接树中最近节点的方式拓展树
 - 考虑机器人的运动执行能力
 - 通过树结构可以直接回溯得到路径



2. RRT(Rapid-Exploring Random Tree)

Algorithm 1: BUILD RRT

Input: q_{init}

Output: T

```
1  $T.init(q_{init});$ 
2 for  $j = 1$  to  $K$  do
3    $q_{rand} \leftarrow \text{RANDOM\_CONFIG}();$ 
4    $\text{EXTEND}(T, q_{rand});$ 
5 end
6 Return  $T$ ;
```

Algorithm 2: EXTEND

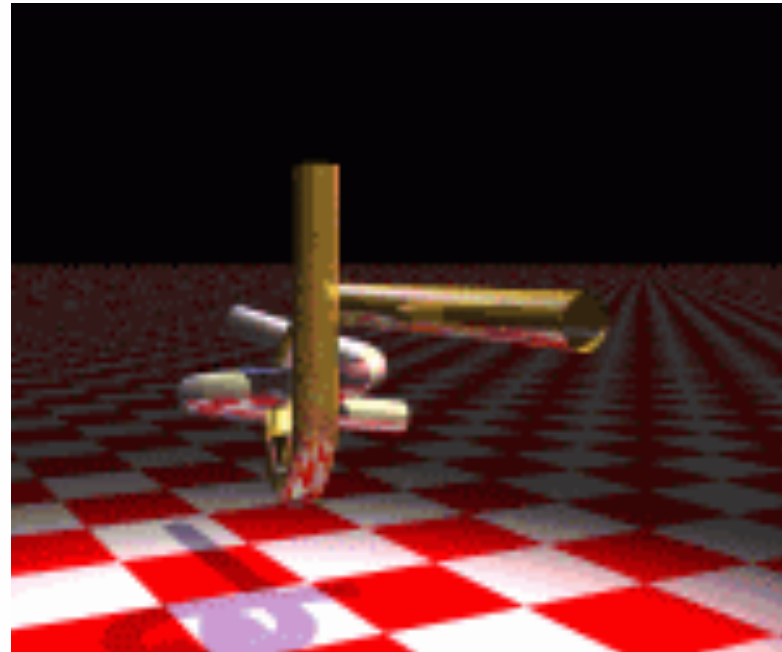
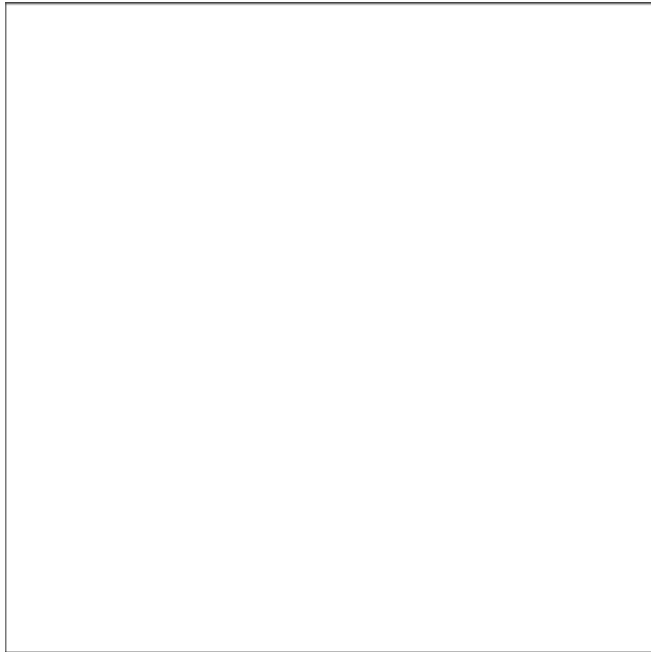
Input: T, q

Output: *Search Result*

```
1  $q_{near} \leftarrow \text{NEAREST\_NEIGHBOR}(q, T);$ 
2 if  $\text{NEW\_CONFIG}(q, q_{near}, q_{new})$  then
3    $T.add\_vertex(q_{new});$ 
4    $T.add\_edge(q_{near}, q_{new});$ 
5   if  $q_{new} = q$  then
6     Return Reached
7   end
8   else
9     Return Advanced
10  end
11 end
12 Return Trapped;
```

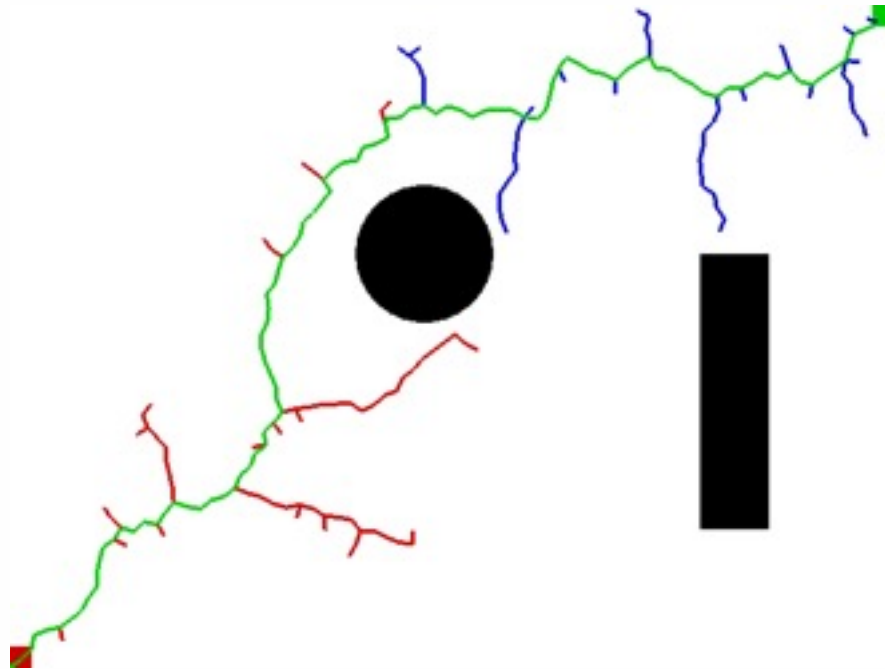
2. RRT(Rapid-Exploring Random Tree)

- 影响规划收敛速度的三个步骤
 - 随机状态的采样
 - 在搜索树中查找与随机状态距离最近的节点，KD树、Hash表等
 - 新生成节点的防碰撞检测



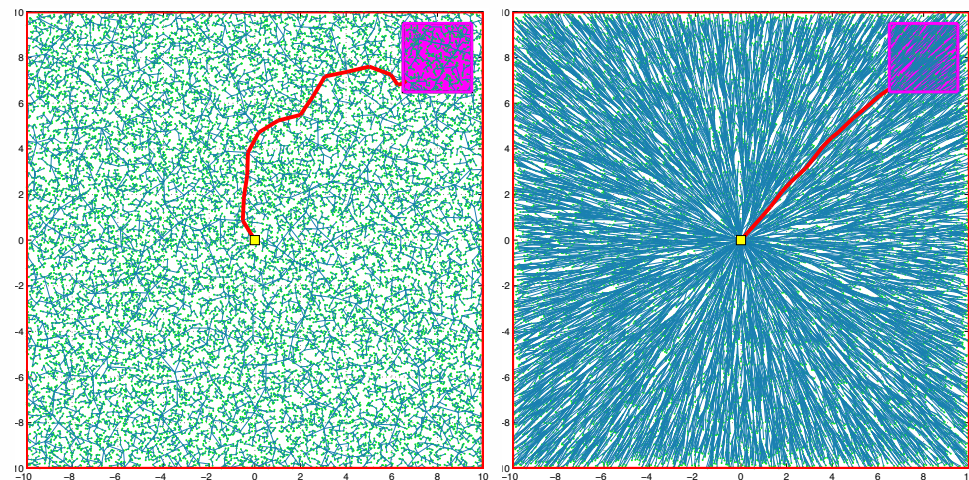
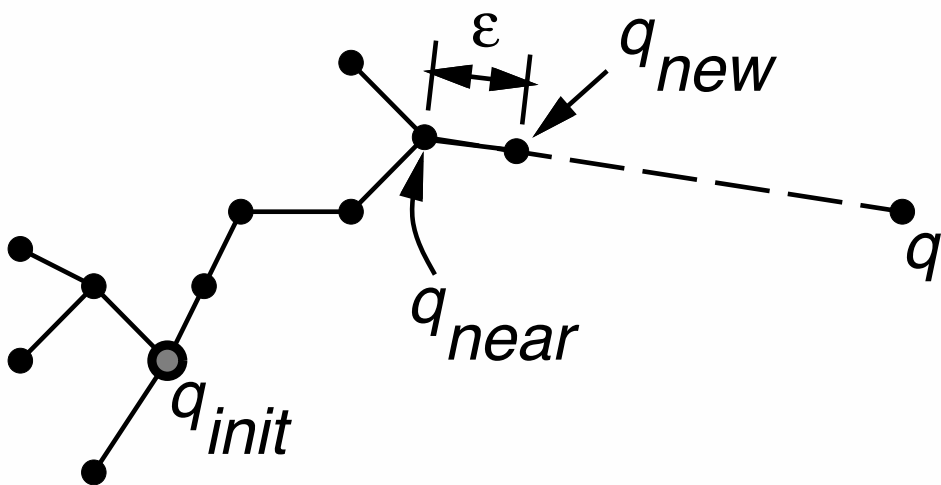
RRT改进1

- 针对问题：RRT扩张偏向状态空间未探测部分，但不是偏向目标点，当环境复杂时计算效率低
- 双向搜索Bidirectional-RRT

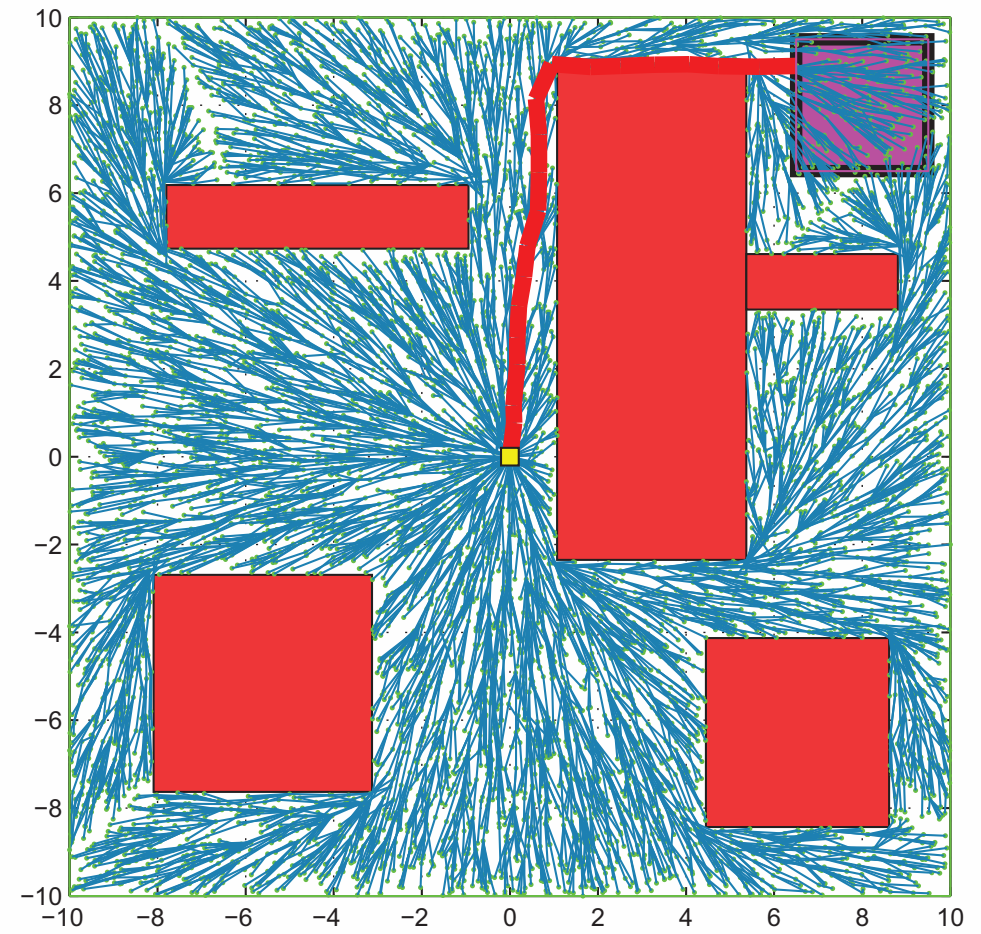
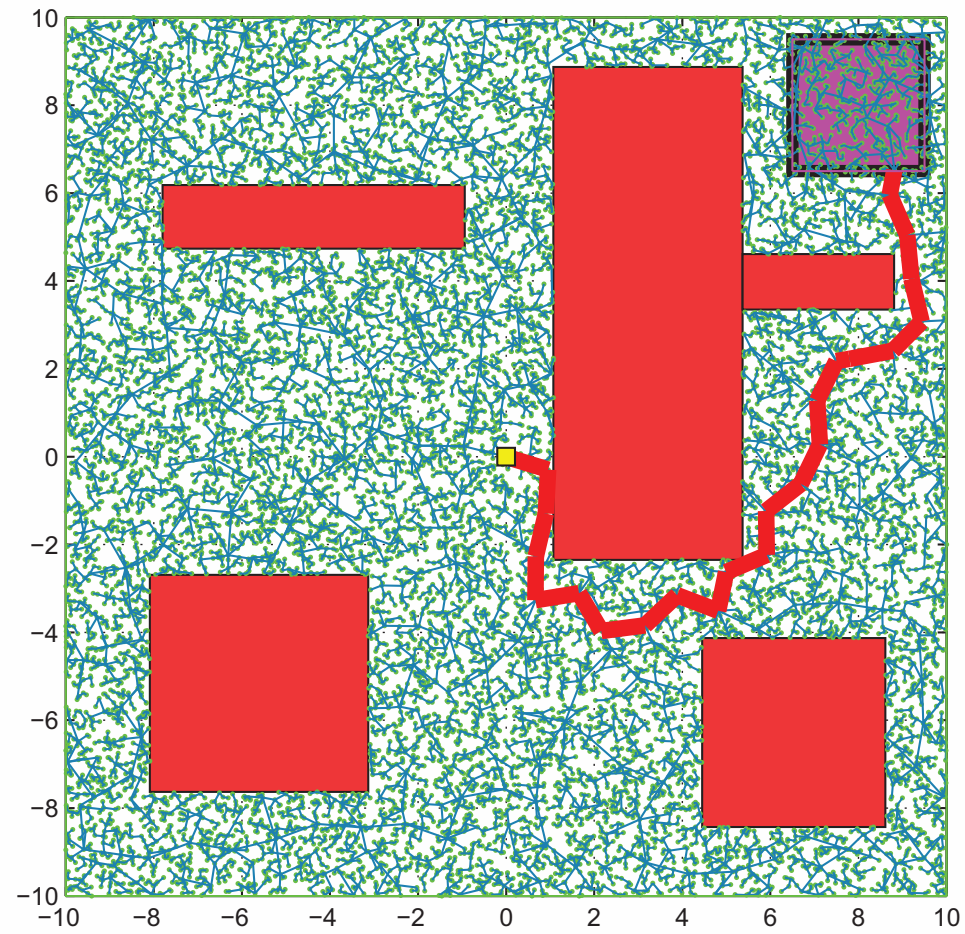


RRT改进2

- 针对问题：随机性小步扩展导致路径曲折，成本高
- RRT*：为实现渐近最优，考虑路径成本
 - 寻找树中新节点邻域内到新节点路径最短的节点，建立连接，加入树集合
 - 对树中新节点邻域内节点进行判断，如果从新节点到该节点形成的路径优于现有树中路径，则将该节点父节点修改为新节点



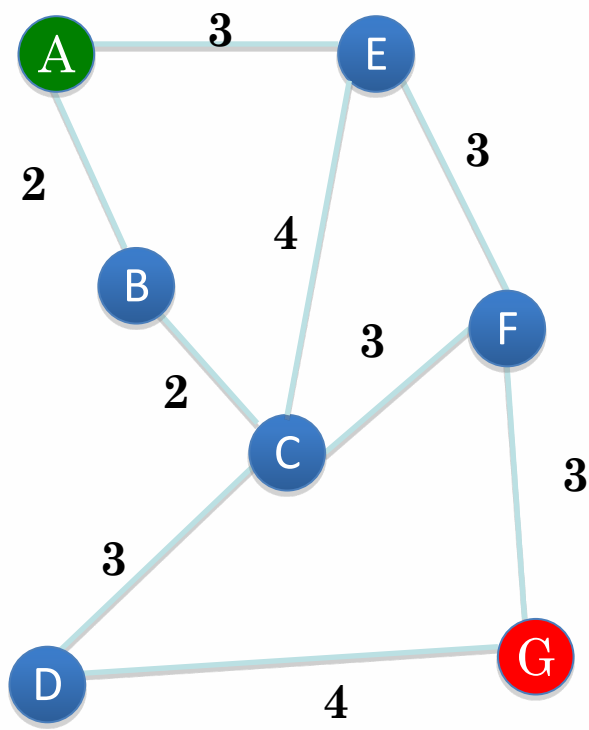
RRT*



4.2.4 最优路径搜索算法

最优路径搜索算法

- 在构建形成的连通图中搜索最优路径

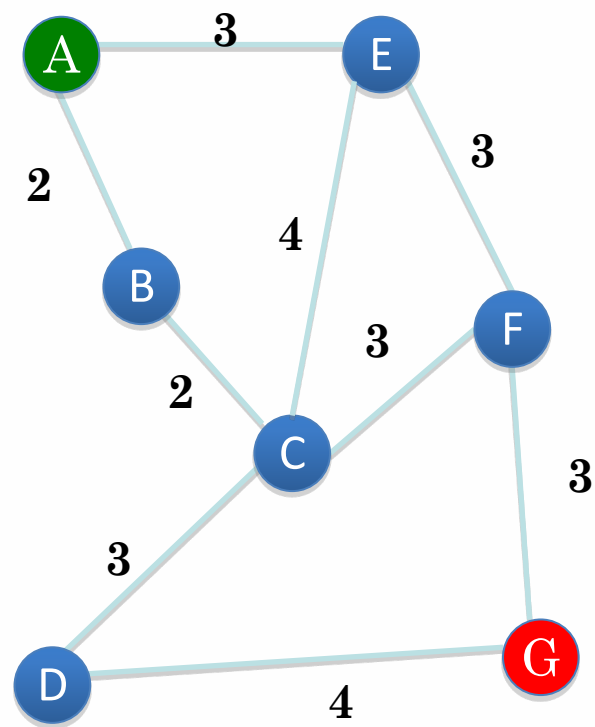


最优路径搜索算法

- 精确算法：生成精确的最优解
 - 深度优先法、广度优先法
 - 遍历获得所有路径后选择最优解
 - 优化：优先级定义的广度优先法、Dijkstra算法
 - 存在问题：耗时，难以满足机器人在线快速规划要求
- 近似算法
 - 启发式搜索算法：A*, D*, Focused D*等
 - 准启发式搜索算法：退火、进化和蚁群优化等

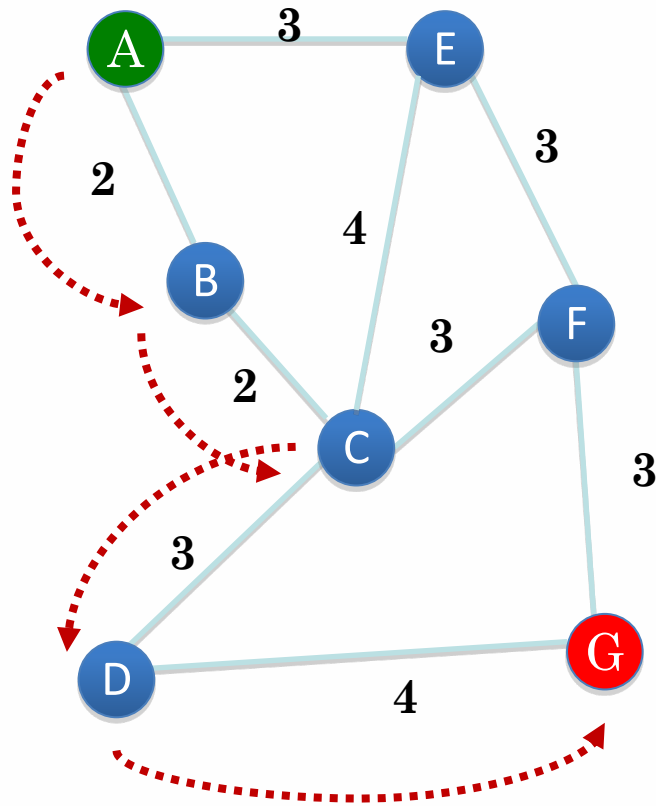
深度优先法（复习）

- 从起始节点开始按深度方式依次探索节点未被访问过的相邻节点
- 在一个相邻节点被访问后，优先访问该节点下一个未被访问相邻节点，直到扩展到图的最深层，即没有可访问的后继相邻节点，或者到达目标节点
- 返回上一层，探索该节点的其他未被访问相邻节点

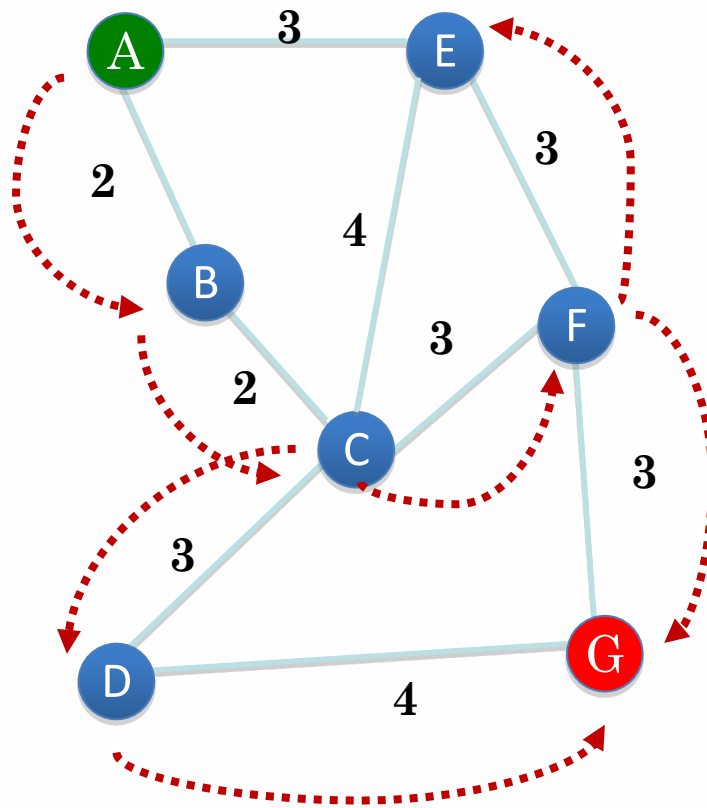


深度优先法

ABCDG 路径长11



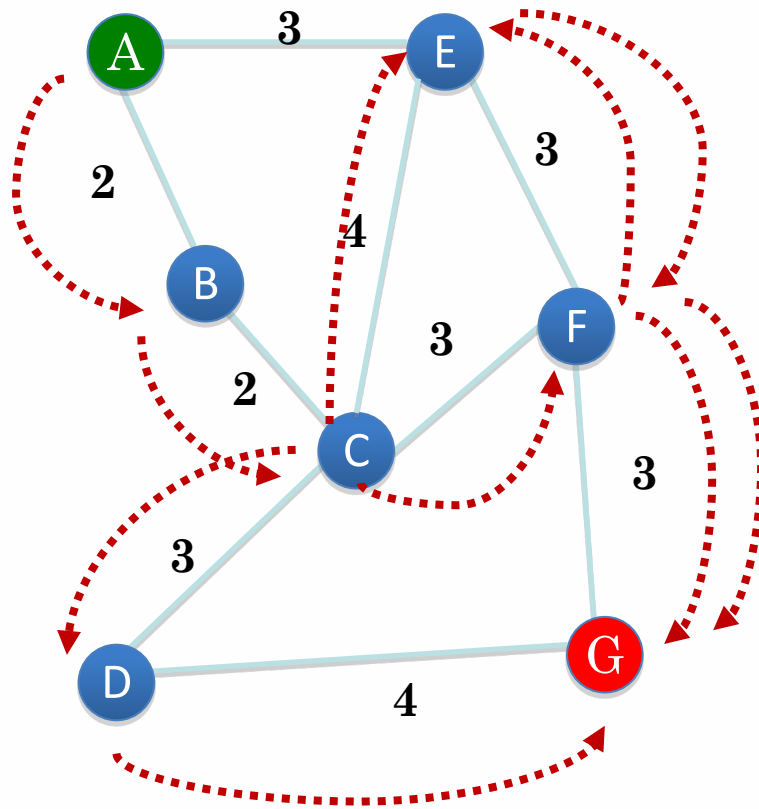
深度优先法



ABCDG 路径长11

ABCFG 路径长10

深度优先法

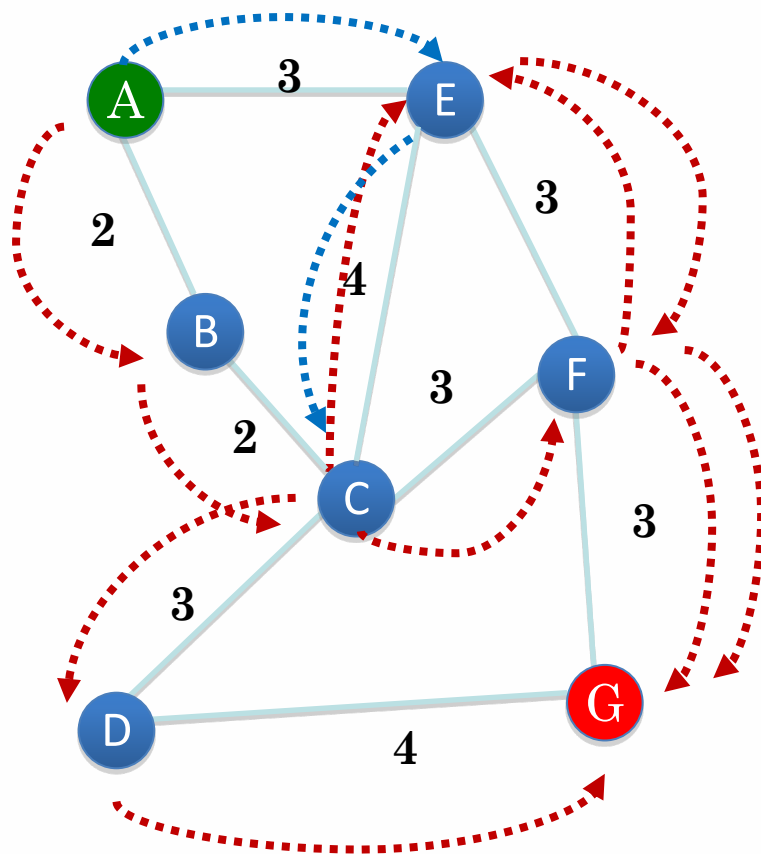


ABCDG 路径长11

ABCFG 路径长10

ABCEFG 路径长14

深度优先法



ABCDG 路径长11

ABCFG 路径长10

ABCEFG 路径长14

AECDG 路径长14

AECFG 路径长13

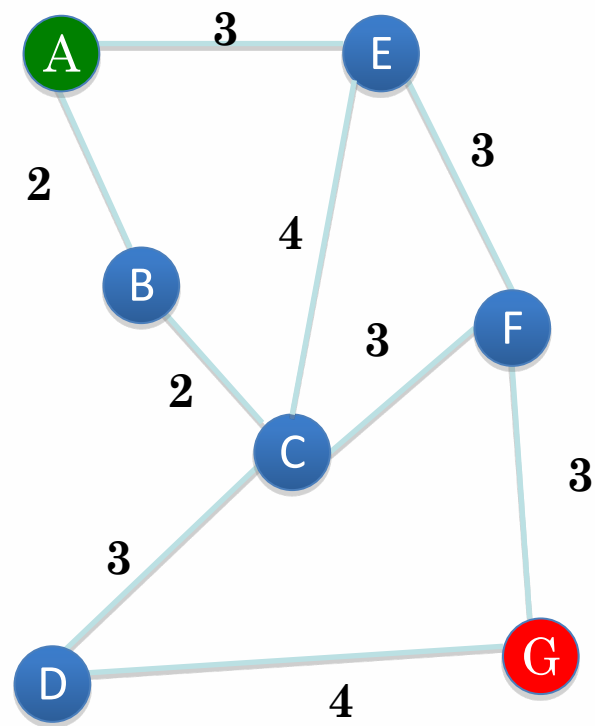
AEFCDG 路径长16

AEFG 路径长9

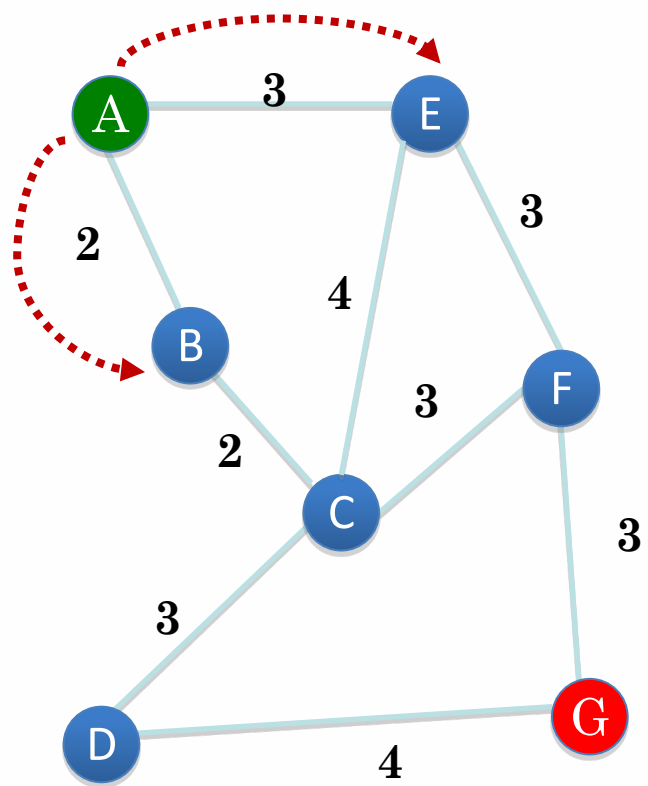
问题：可能会重新访问已经访问过的节点，或者进入冗余路径

优先级定义的广度优先法（复习）

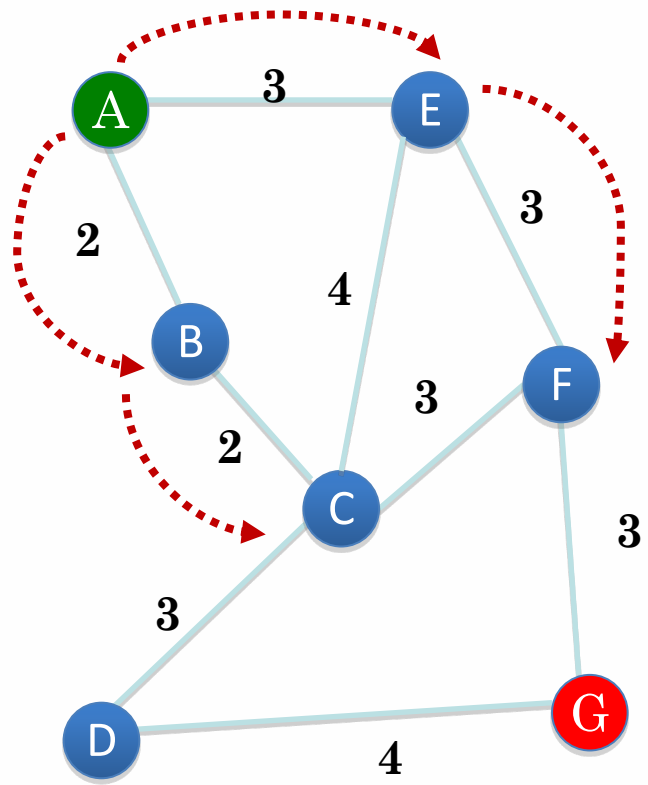
- 从起始节点开始访问该节点所有未曾访问的相邻节点
- 然后分别从这些相邻节点出发依次访问它们的相邻节点，访问优先级为先被访问的节点的相邻节点优先于后被访问的节点的相邻节点，直到所有被访问节点的相邻节点都被访问到



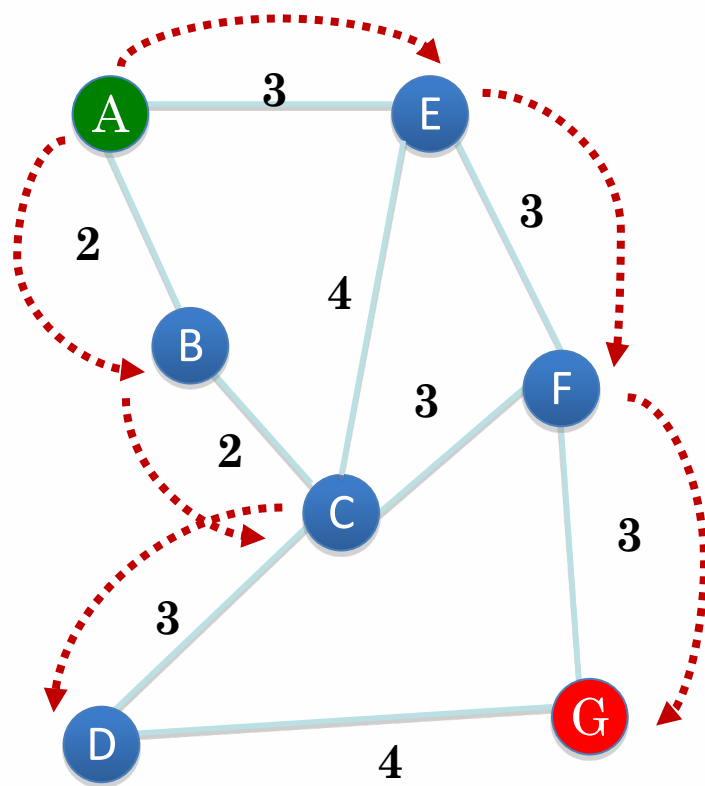
优先级定义的广度优先法



优先级定义的广度优先法

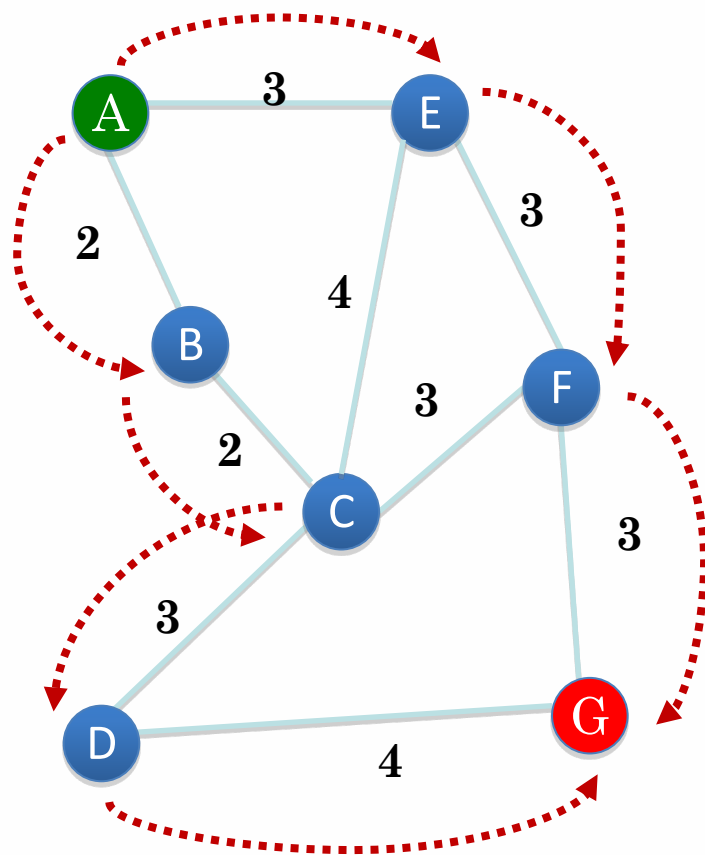


优先级定义的广度优先法



AEFG 路径长9

优先级定义的广度优先法



AEFG 路径长9

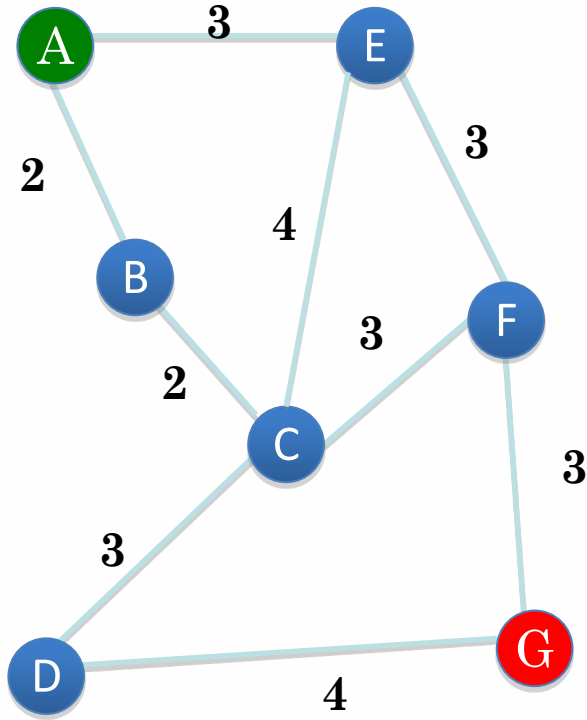
ABCDG 路径长11

由于广度优先法定义路访问优先级，因此不会遍历所有路径，但可能会导致搜索不到最短路径。

Dijkstra算法（复习）

- 采用优先级定义的广度优先搜索思想
 - 在有向图中以起始点为中心按路径长度递增方式层层向外扩展，直到扩展到终止点
- 设置两个集合S和T， S存放已经找到最短路径的顶点， T是尚未确定路径的顶点集合，同时也描述了起始点经过集合S中顶点到该点的最短路径及长度。
 - 每次更新时，从T中找出路径最短的点加入到集合S， T中顶点最短路径及长度则根据加入点作为中间点后起始点到该点距离是否减小来决定是否更新

Dijkstra算法



$S = \{A \rightarrow A = 0\}$

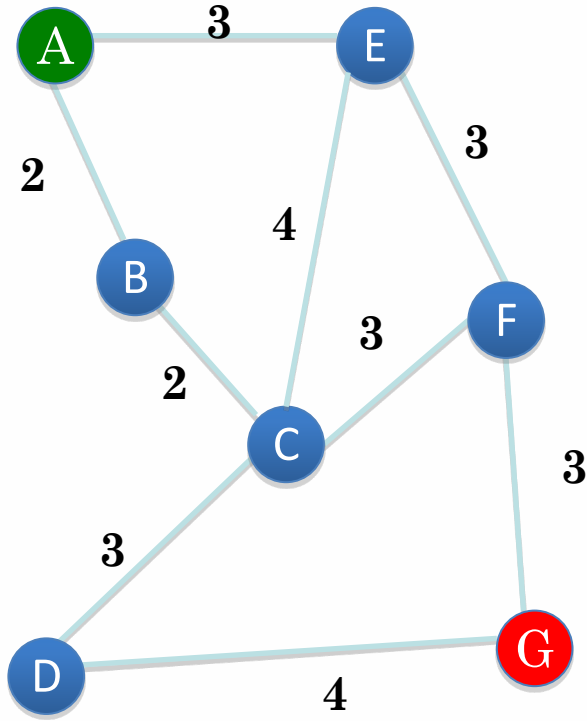
$T = \{A \rightarrow B = 2, A \rightarrow C = \infty, A \rightarrow D = \infty, A \rightarrow E = 3, A \rightarrow F = \infty, A \rightarrow G = \infty\}$



$S = \{A \rightarrow A = 0, A \rightarrow B = 2\}$

$T = \{A \rightarrow C = 4 \text{ (路径为ABC)}, A \rightarrow D = \infty, A \rightarrow E = 3, A \rightarrow F = \infty, A \rightarrow G = \infty\}$

Dijkstra算法



问题规模大时搜索耗时

$S = \{A \rightarrow A = 0, A \rightarrow B = 2, A \rightarrow E = 3\}$

$T = \{A \rightarrow C = 4(\text{路径为ABC}), A \rightarrow D = \infty, A \rightarrow F = 6(\text{路径为AEF}), A \rightarrow G = \infty\}$

$S = \{A \rightarrow A = 0, A \rightarrow B = 2, A \rightarrow E = 3, A \rightarrow C = 4(\text{路径为ABC})\}$

$T = \{A \rightarrow D = 7(\text{路径为ABCD}), A \rightarrow F = 6(\text{路径为AEF}), A \rightarrow G = \infty\}$



.....

启发式搜索算法：A*

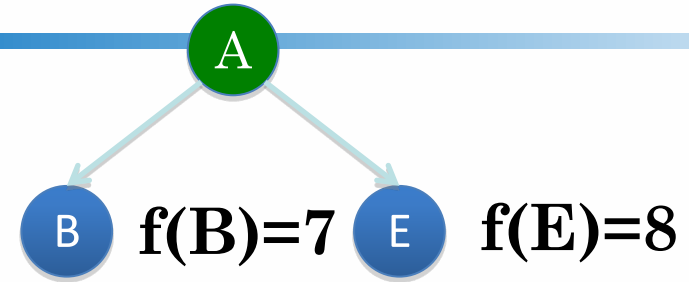
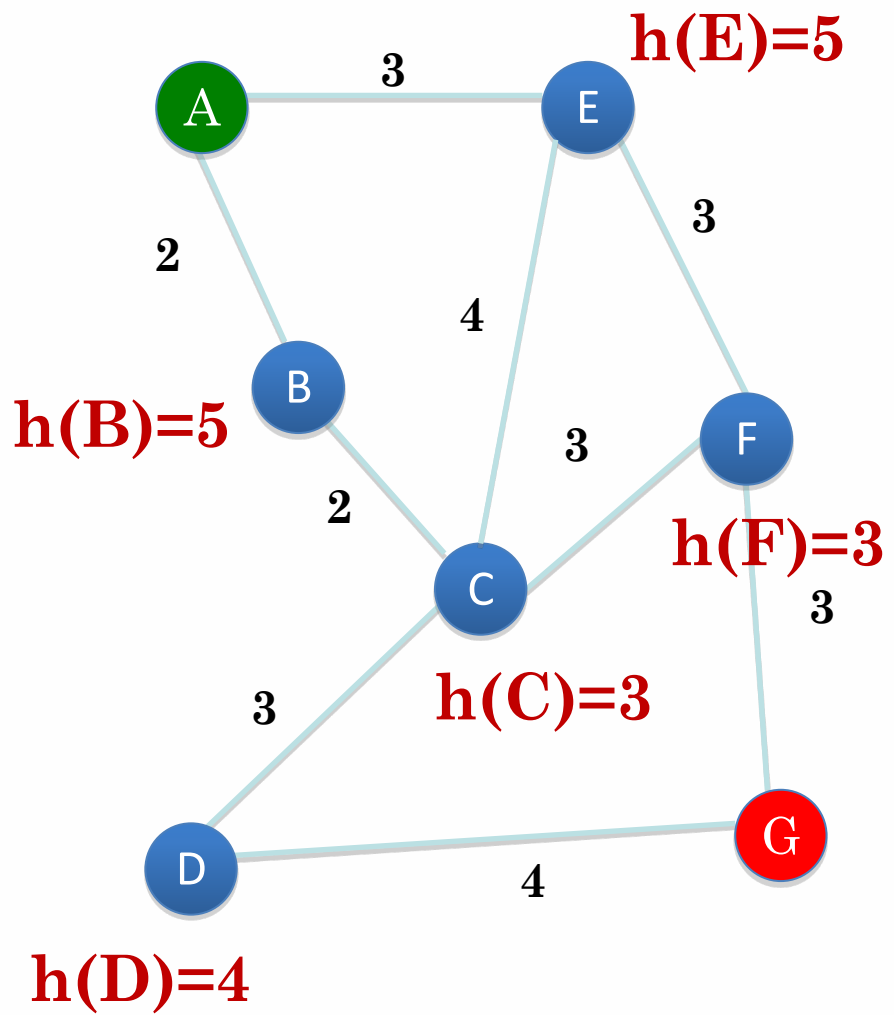
- 基于优先级定义的广度优先搜索
- 根据**启发式评估函数**在连通图中寻找最优路径
 - 当选择下一个探索结点时，通过启发式评估函数进行评估，选择路径代价最小的结点作为下一步探索结点而跳转其上

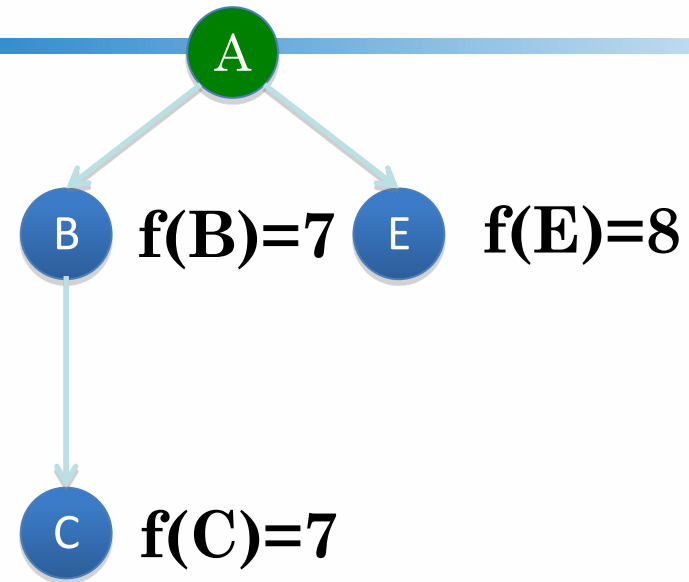
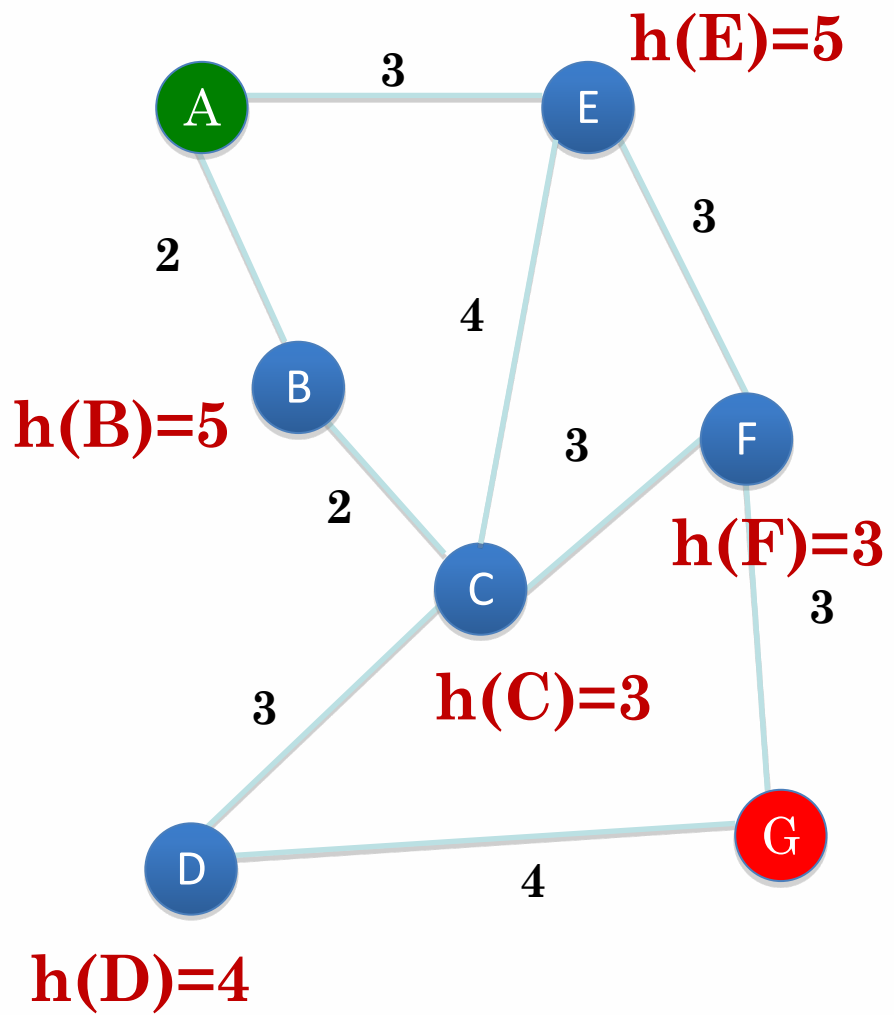
评估函数 $f(n) = g(n) + h(n)$

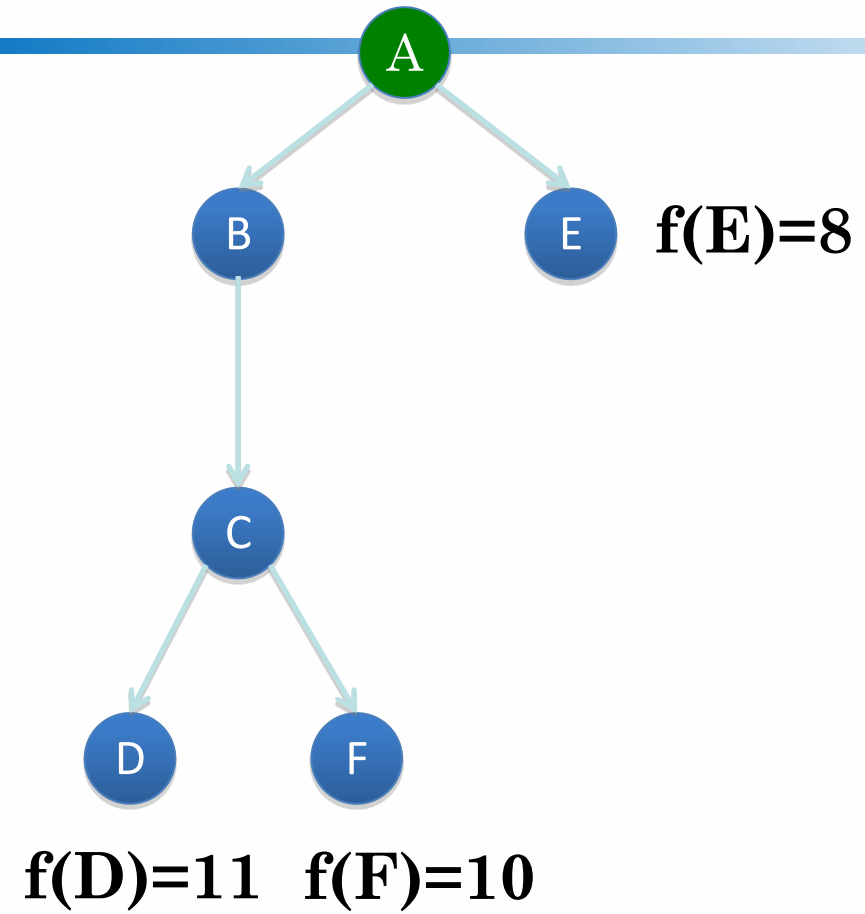
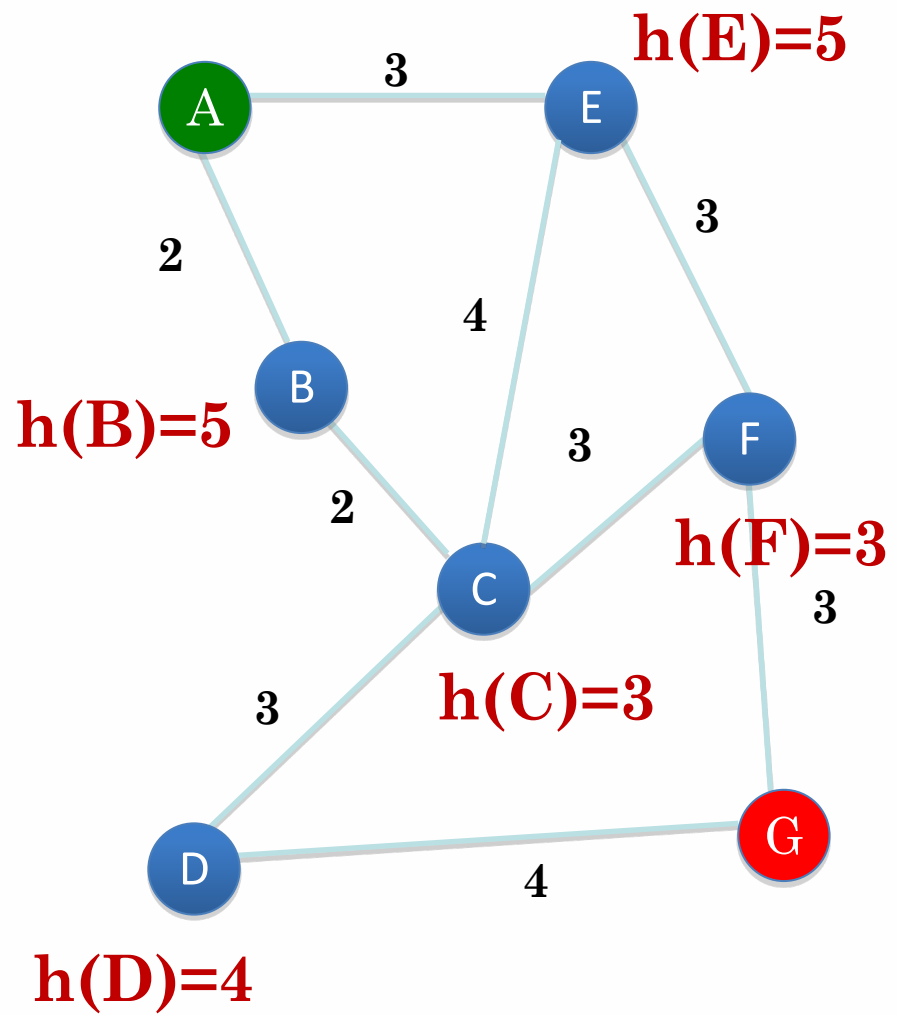
n 表示节点

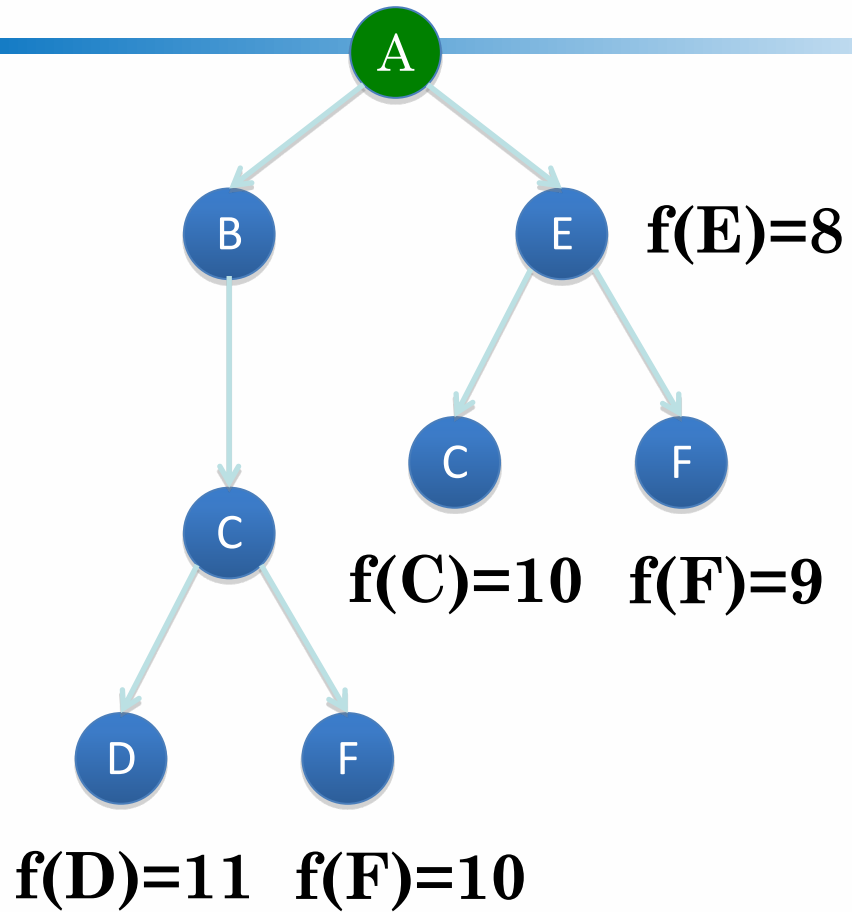
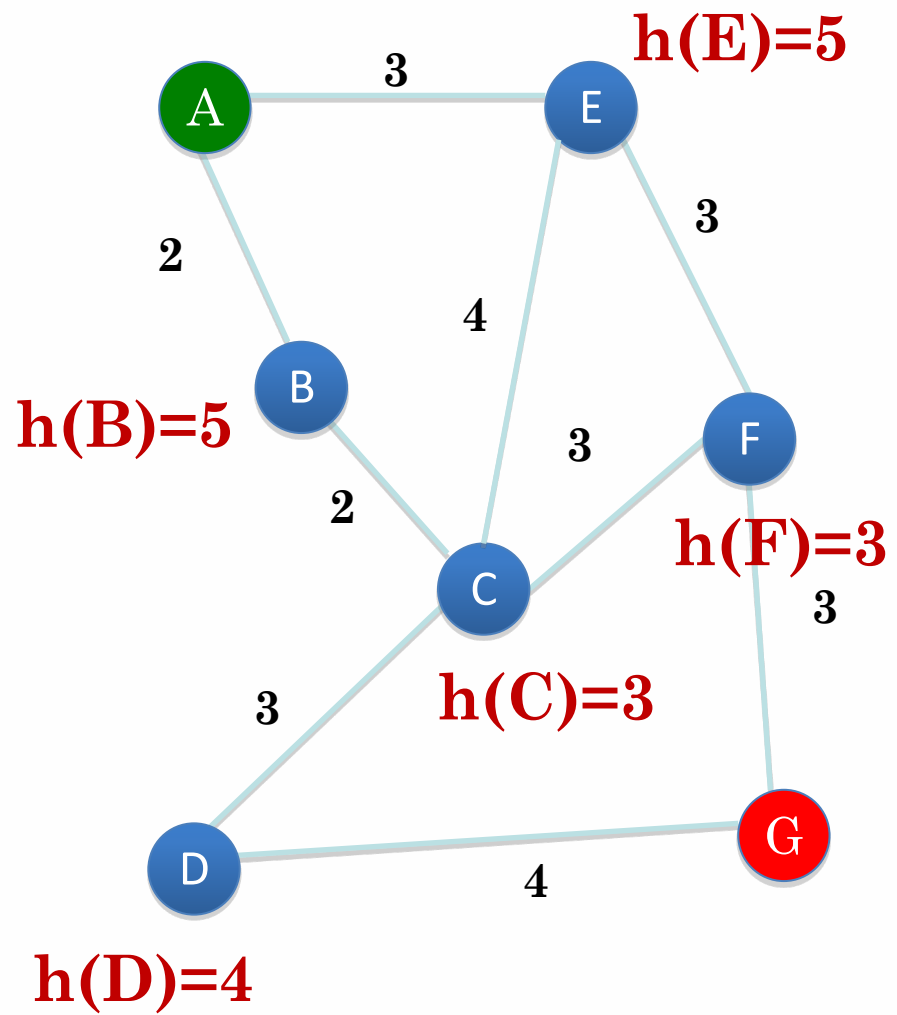
$g(n)$ 表示从起始点到节点 的实际代价

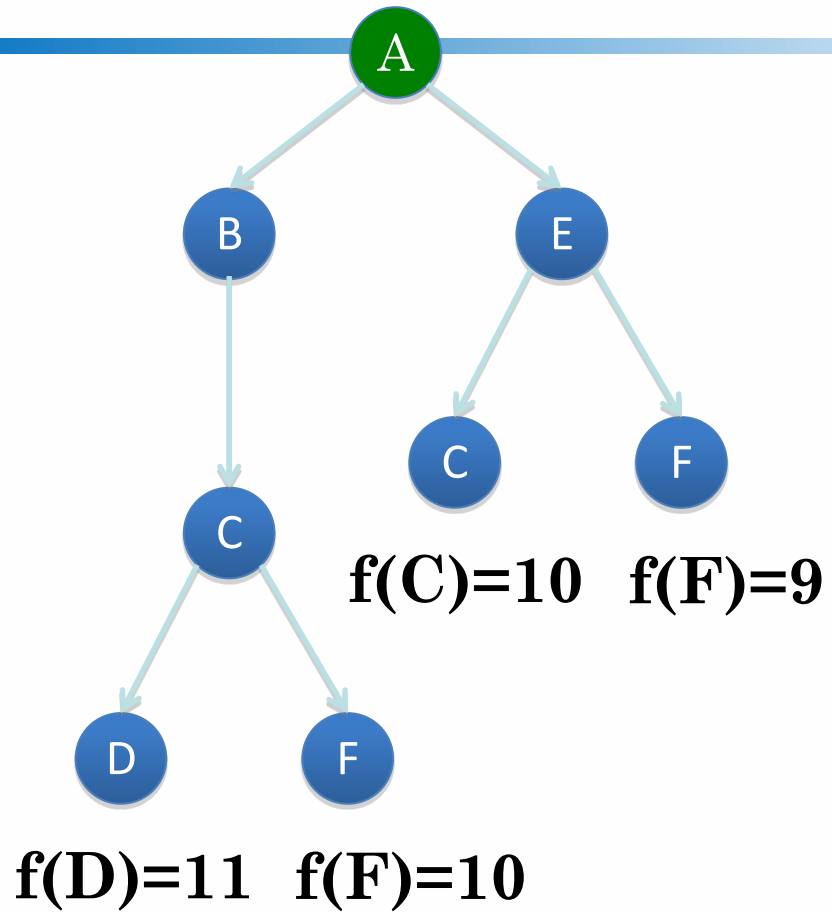
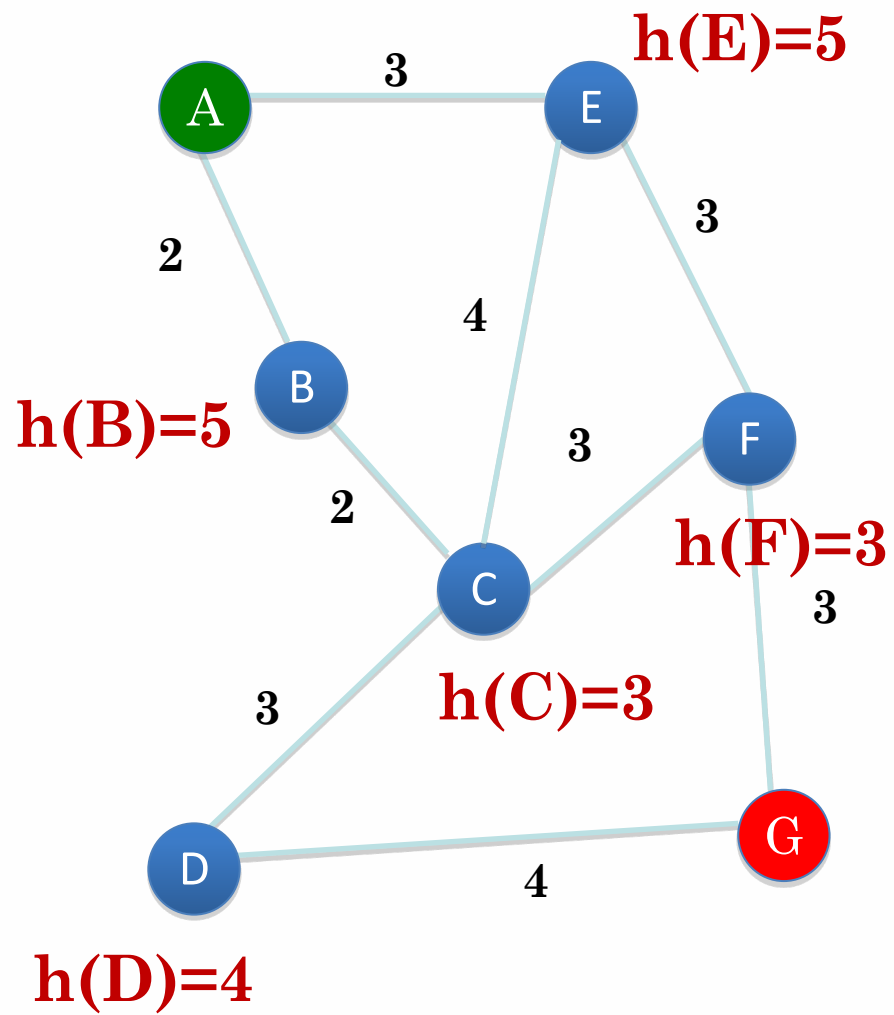
$h(n)$ 为从节点 到目标点的最佳路径的估计代价

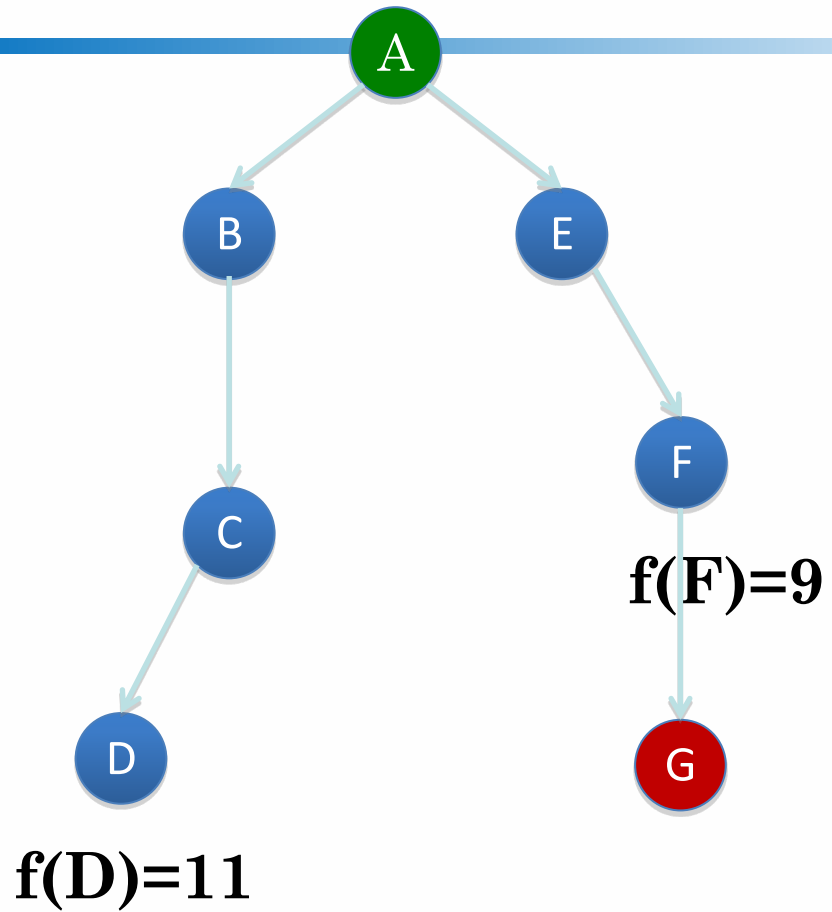
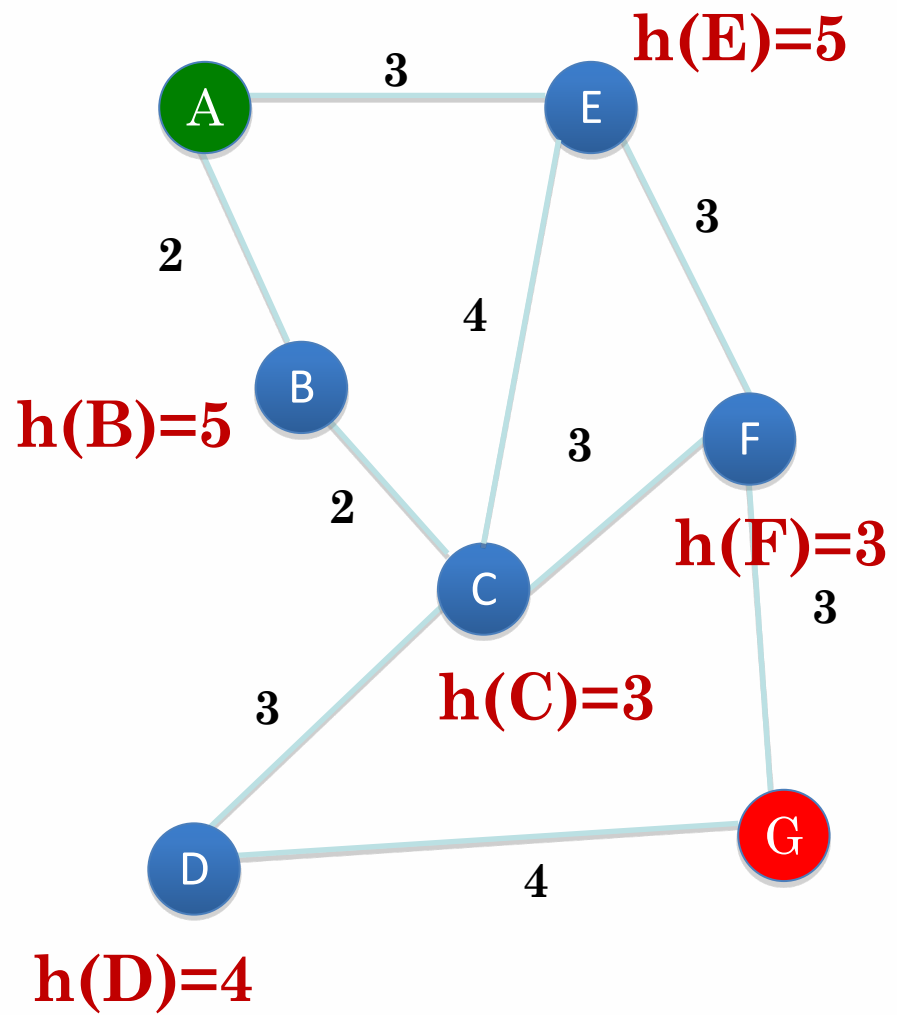


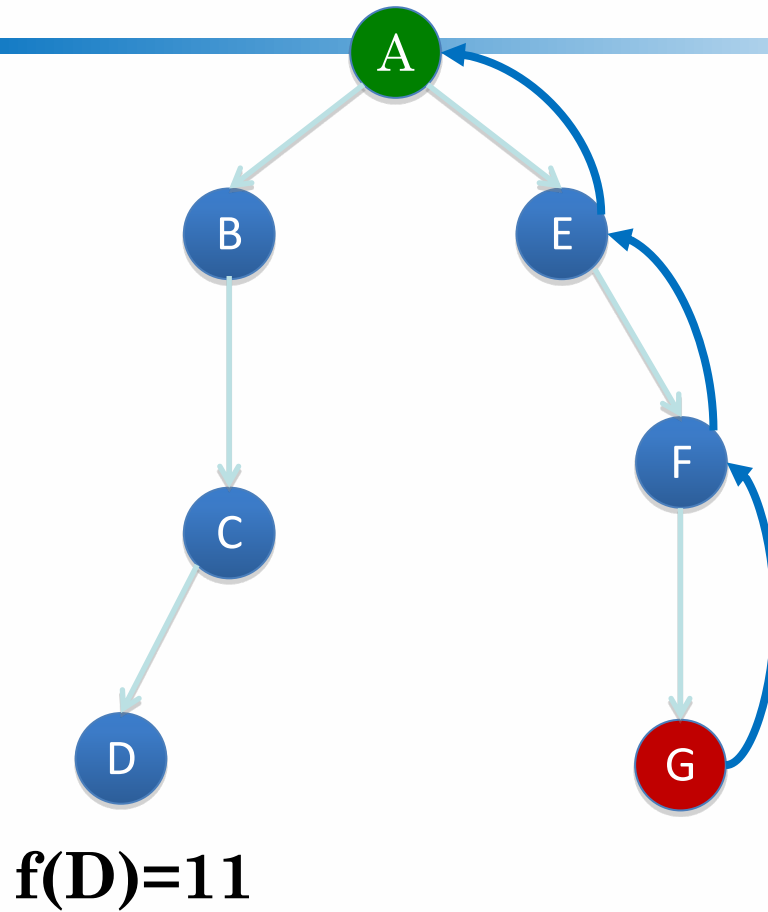
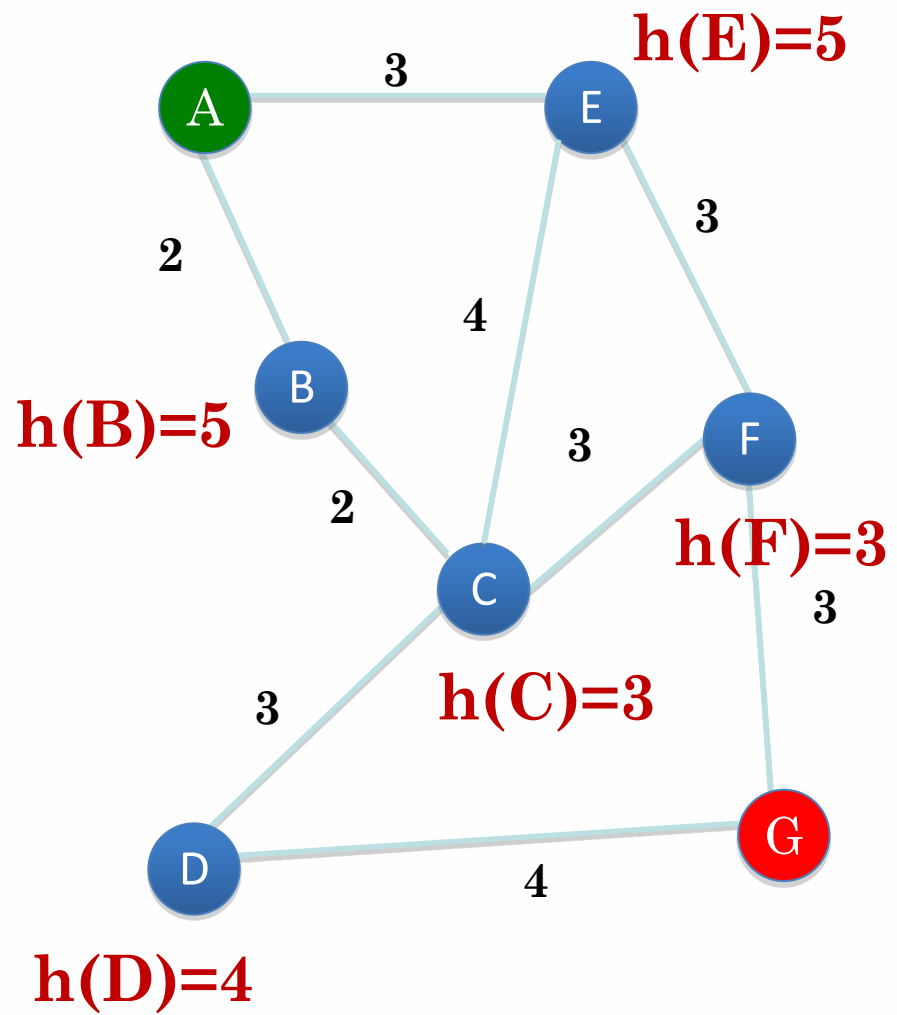




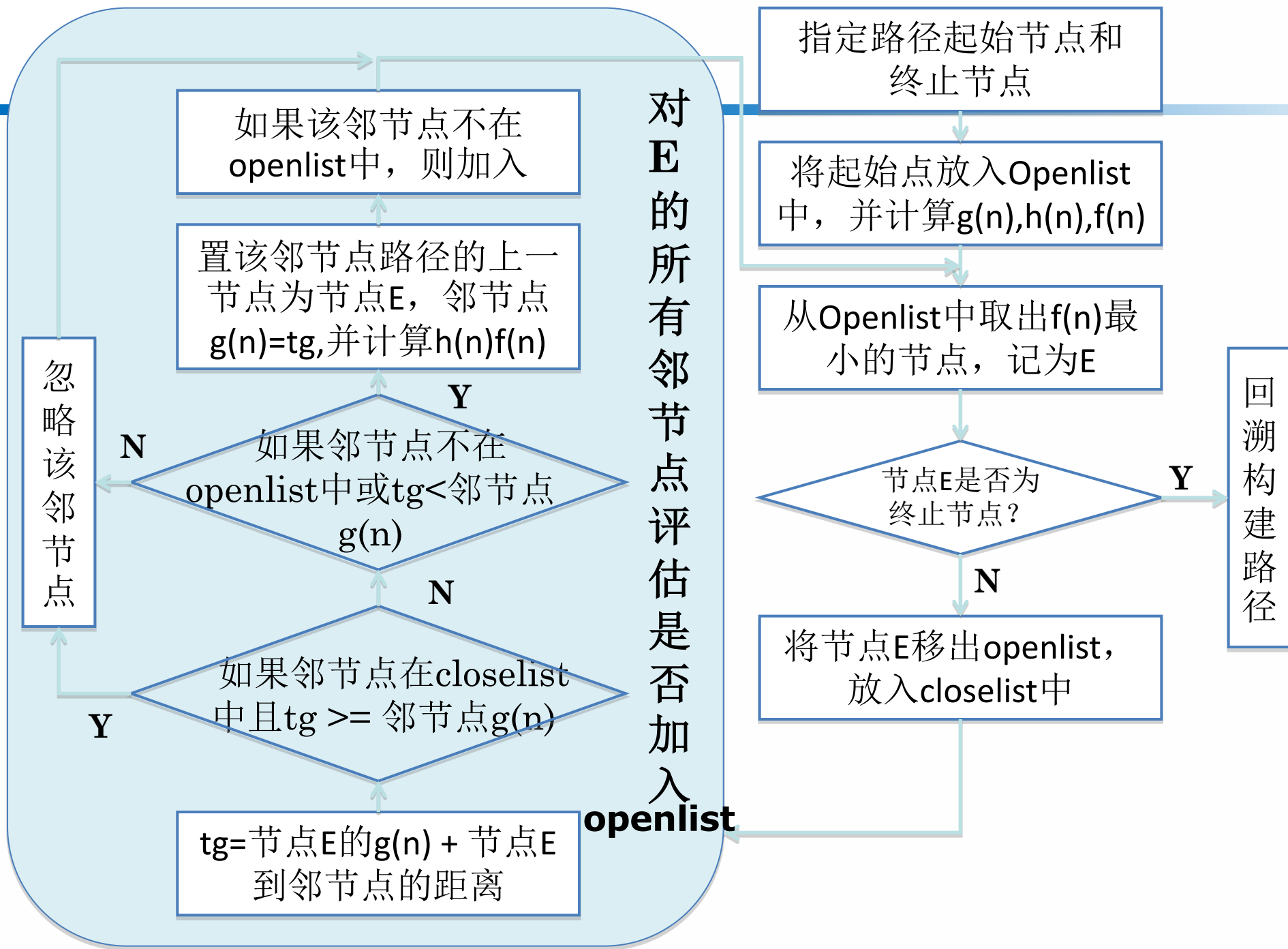






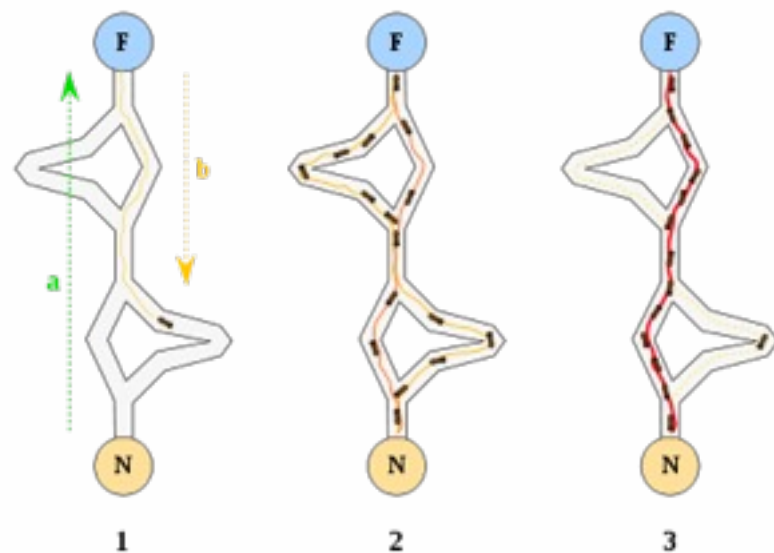
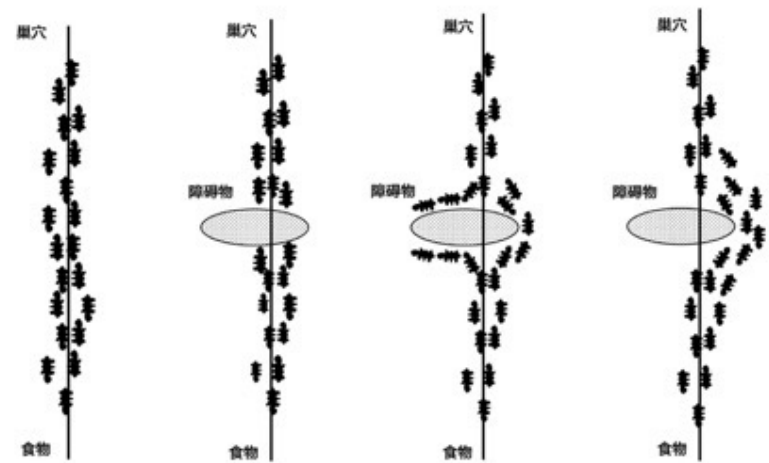


对E的所有邻节点评估是否加入



准启发式搜索算法： 蚁群算法

- 1992年由意大利科学家M.Dorigo提出
- 来源于模拟蚂蚁觅食过程中寻找路径的行为
- 基本思想：
 - 用一个蚂蚁的行走路径表示待优化问题的一个可行解
 - 用整个蚂蚁群体的所有路径作为待优化问题的解空间
 - 用蚂蚁群体收敛选择的路径作为问题的优化解



蚁群算法

- 实现方式：
 - 每个蚂蚁在经过的路径上释放信息素，称为正反馈

$$\tau_{ij}(t) \leftarrow \tau_{ij}(t) + \Delta\tau$$

- $\Delta\tau$ 可以是经过该路段留下的即时信息素，也可以是到达目标点后对整段路径留下的信息素
- 信息素与路径长度成反比
- 为避免早熟收敛，信息素采用挥发机制

$$\tau \leftarrow (1 - \rho)\tau, \rho \in [0, 1]$$

蚁群算法

- 实现方式：
 - 信息素计算总表达式

$$\tau_{ij}(t+n) = (1-\rho)^n \tau_{ij}(t) + \Delta\tau_{ij}(t+n)$$
$$\Delta\tau_{ij}(t+n) = \sum_{k=1}^m \Delta\tau_{ij}^k(t+n)$$

m 为蚂蚁总数

k 为蚂蚁编号

蚁群算法

- 实现方式:

- 每个蚂蚁根据后续可选边的评估概率地决策

$$p_{ij}^k(t) = \begin{cases} \frac{a_{ij}(t)}{\sum_{l \in N_i^k} a_{il}(t)} & \text{if } j \in N_i^k \\ 0 & \text{if not} \end{cases}$$

k 表示蚂蚁编号, N_i^k 为第 k 个蚂蚁当点节点 i 的相邻节点集合, $a_{ij}(t)$ 为节点 i 的路径评估值

$$a_{ij} = \frac{\tau_{ij}^\alpha(t) \eta_{ij}^\beta(t)}{\sum_{l \in N_i^k} \tau_{il}^\alpha(t) \eta_{il}^\beta(t)}, \quad \forall j \in N_i^k$$

- α 为信息启发式因子, 反应路径上信息素对蚂蚁选择路径的影响程度, 值越大蚂蚁之间的协作性越强
- $\eta_{ij}(t)$ 为目标启发式函数, 表示从节点 i 到节点 j 的期望程度, 通常取 i 和 j 之间路径长度的反比
- β 为期望因子, 表示启发式的重要性
- 需要在启发式和信息素两者之间做权衡

蚁群算法

- 通过局部探测加记忆机制来搜索可行解
- 优点：
 - 问题的图表示随着最优化过程同步变化，因此可以适应问题的动态性
 - 具有稀疏分布计算架构，可实现并行计算
- 缺点：
 - 计算量大，求解需要时间长
 - 参数需要依靠经验反复调试，不同环境适配不同参数